

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

26 个 Kernel · 10 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 6 月 28 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (共 26 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 2D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89 //
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
102 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
103 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
104 //
105 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
106 //
107 // =====
108 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
109 // =====
110 // 面试要点:
111 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
112 //   memory
113 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
114 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
115 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
116 //
117 #include <algorithm>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127
128 #define WARP_SIZE 32
129 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
130 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
131 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
132
133 // =====
134 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
135 // =====
136
137 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
138 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
139 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
140 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
141 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
142 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
143 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
144 //
145 // 初始: 每个 lane 持有自己的值 v0..v7
146 // lane:  0    1    2    3    4    5    6    7
147 // val:  v0   v1   v2   v3   v4   v5   v6   v7
148 //
149 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
150 //
151 //      ┌───────────┐
152 // 对:   (0,4) (1,5) (2,6) (3,7)
153 //
154 // lane:  0    1    2    3    4    5    6    7
155 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
156 //
157 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
158 //
159 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
160 // 对:   (0,2) (1,3) (4,6) (5,7)
161 //      ─── 前4个一组 ───      ─── 后4个一组 ───
162 //
163 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
164 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
165 //      = v0+v2+v4+v6 ... (逐步归约)
166 //
167 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
168 //
169 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
170 // 对:   (0,1) (2,3) (4,5) (6,7)
171 //
172 // lane:  0    1    2    3    4    5    6    7
173 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
174 //
175 // mask=0: 循环终止, 归约完成。
176 //
177 // 关键性质:
178 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
179 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
180 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
181 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
182 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
183 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
184 template <const int kWarpSize = WARP_SIZE, typename T = float>
185 __device__ __forceinline__ T warp_reduce_sum(T val) {
186 #pragma unroll
187   for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
188     val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
189   }
190   return val;
191 }

```

```

189
190 // Warp Reduce Max — generic
191 template <const int kWarpSize = WARP_SIZE, typename T = float>
192 __device__ __forceinline__ T warp_reduce_max(T val) {
193 #pragma unroll
194     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
195         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
196     }
197     return val;
198 }
199
200 // =====
201 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
202 // =====
203
204 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
205 // 两级归约流程:
206 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
207 // 2. warp leader (lane=0) 写入 shared memory
208 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
209 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
210 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
211 template <const int NUM_THREADS = 256>
212 __device__ float block_reduce_sum(float val) {
213     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
214     int warp = threadIdx.x / WARP_SIZE;
215     int lane = threadIdx.x % WARP_SIZE;
216     static __shared__ float shared[NUM_WARPS];
217
218     float value = warp_reduce_sum<WARP_SIZE>(val);
219     if (lane == 0)
220         shared[warp] = value;
221     __syncthreads();
222     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
223     value = warp_reduce_sum<NUM_WARPS>(value);
224     // 关键: broadcast 结果到所有线程, 后续用 result
225     // 做除法等操作时所有线程都能拿到
226     value = __shfl_sync(0xffffffff, value, 0, 32);
227     return value;
228 }
229
230 // Block Reduce Max — FP32 (增强版, 带 broadcast)
231 template <const int NUM_THREADS = 256>
232 __device__ float block_reduce_max(float val) {
233     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
234     int warp = threadIdx.x / WARP_SIZE;
235     int lane = threadIdx.x % WARP_SIZE;
236     static __shared__ float shared[NUM_WARPS];
237
238     float value = warp_reduce_max<WARP_SIZE>(val);
239     if (lane == 0)
240         shared[warp] = value;
241     __syncthreads();
242     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
243     value = warp_reduce_max<NUM_WARPS>(value);
244     value = __shfl_sync(0xffffffff, value, 0, 32);
245     return value;
246 }
247
248 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
249 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
250 // 跨 block 求和的常见模式, 适合 N 较大时使用;
251 // Grid: ((N + 255) / 256, 1, 1)

```

```

252 // Block: (256, 1, 1)
253 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
254 template <const int NUM_THREADS = 256>
255 __global__ void block_reduce_all(float *a, float *y, int N) {
256     int tid = threadIdx.x;
257     int idx = blockIdx.x * NUM_THREADS + tid;
258     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
259     __shared__ float reduce_smem[NUM_WARPS];
260
261     float val = (idx < N) ? a[idx] : 0.0f;
262     int warp = tid / WARP_SIZE;
263     int lane = tid % WARP_SIZE;
264
265     val = warp_reduce_sum<WARP_SIZE>(val);
266     if (lane == 0)
267         reduce_smem[warp] = val;
268     __syncthreads();
269
270     val = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
271     if (warp == 0)
272         val = warp_reduce_sum<NUM_WARPS>(val);
273     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
274         atomicAdd(y, val);
275 }
276
277 // ---- Dot Product: y = sum(a[i] * b[i]) ----
278 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
279 // Grid: ((N + 255) / 256, 1, 1)
280 // Block: (256, 1, 1)
281 // source: LeetCUDA/kernels/dot-product/dot_product.cu
282 template <const int NUM_THREADS = 256>
283 __global__ void dot(float *a, float *b, float *y, int N) {
284     int tid = threadIdx.x;
285     int idx = blockIdx.x * NUM_THREADS + tid;
286     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
287     __shared__ float reduce_smem[NUM_WARPS];
288
289     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
290     int warp = tid / WARP_SIZE;
291     int lane = tid % WARP_SIZE;
292
293     prod = warp_reduce_sum<WARP_SIZE>(prod);
294     if (lane == 0)
295         reduce_smem[warp] = prod;
296     __syncthreads();
297
298     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
299     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
300         prod = warp_reduce_sum<NUM_WARPS>(prod);
301     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
302         atomicAdd(y, prod);
303 }
304
305 // Dot Product + float4
306 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
307 // Block: (64, 1, 1), 256/4=64
308 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
309 // source: LeetCUDA/kernels/dot-product/dot_product.cu
310 template <const int NUM_THREADS = 256 / 4>
311 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
312     int tid = threadIdx.x;
313     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
314     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;

```

```

315 __shared__ float reduce_smem[NUM_WARPS];
316
317 float4 reg_a = FLOAT4(a[idx]);
318 float4 reg_b = FLOAT4(b[idx]);
319 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
320                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
321                          : 0.0f;
322 int warp = tid / WARP_SIZE;
323 int lane = tid % WARP_SIZE;
324
325 prod = warp_reduce_sum<WARP_SIZE>(prod);
326 if (lane == 0)
327     reduce_smem[warp] = prod;
328 __syncthreads();
329
330 prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
331 if (warp == 0)
332     prod = warp_reduce_sum<NUM_WARPS>(prod);
333 if (tid == 0)
334     atomicAdd(y, prod);
335 }
336
337
338 // =====
339 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
340 // =====
341 // 面试要点:
342 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
343 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
344 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
345
346 // ---- ReLU: y = max(0, x) ----
347 // Grid: ((N + 255) / 256, 1, 1)
348 // Block: (256, 1, 1)
349 // source: LeetCUDA/kernels/relu/relu.cu
350 __global__ void relu(float *x, float *y, int N) {
351     int idx = blockIdx.x * blockDim.x + threadIdx.x;
352     if (idx < N)
353         y[idx] = fmaxf(0.0f, x[idx]);
354 }
355
356 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
357 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
358 // Grid: ((N + 255) / 256, 1, 1)
359 // Block: (64, 1, 1)
360 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
361 // source: LeetCUDA/kernels/relu/relu.cu
362 __global__ void relu_vec4(float *x, float *y, int N) {
363     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
364     if (idx < N) {
365         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
366         float4 reg_y;
367         reg_y.x = fmaxf(0.0f, reg_x.x);
368         reg_y.y = fmaxf(0.0f, reg_x.y);
369         reg_y.z = fmaxf(0.0f, reg_x.z);
370         reg_y.w = fmaxf(0.0f, reg_x.w);
371         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
372     }
373 }
374
375 // ---- Elementwise Add: c = a + b ----
376 // Grid: ((N + 255) / 256, 1, 1)
377 // Block: (256, 1, 1)

```

```

378 // source: LeetCUDA/kernels/elementwise/elementwise.cu
379 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
380     int idx = blockIdx.x * blockDim.x + threadIdx.x;
381     if (idx < N)
382         c[idx] = a[idx] + b[idx];
383 }
384
385 // Elementwise Add + float4 向量化
386 // Grid: ((N + 255) / 256, 1, 1)
387 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
388 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
389 // source: LeetCUDA/kernels/elementwise/elementwise.cu
390 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
391     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
392     if ((idx + 3) < N) {
393         float4 reg_a = FLOAT4(a[idx]);
394         float4 reg_b = FLOAT4(b[idx]);
395         float4 reg_c;
396         reg_c.x = reg_a.x + reg_b.x;
397         reg_c.y = reg_a.y + reg_b.y;
398         reg_c.z = reg_a.z + reg_b.z;
399         reg_c.w = reg_a.w + reg_b.w;
400         FLOAT4(c[idx]) = reg_c;
401     } else if (idx < N) {
402         for (int i = 0; (idx + i) < N; i++) {
403             c[idx + i] = a[idx + i] + b[idx + i];
404         }
405     }
406 }
407
408 // ---- Histogram: y[a[i]]++ ----
409 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
410 // Grid: ((N + 255) / 256, 1, 1)
411 // Block: (256, 1, 1)
412 // source: LeetCUDA/kernels/histogram/histogram.cu
413 __global__ void histogram(int *a, int *y, int N) {
414     int idx = blockIdx.x * blockDim.x + threadIdx.x;
415     if (idx < N)
416         atomicAdd(&(y[a[idx]]), 1);
417 }
418
419 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
420 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
421 //
422 // 面试要点:
423 //   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
424 //     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
425 //   - 数学公式 (5 步推导):
426 //     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
427 //         L_max = max(LSE_1, LSE_2)
428 //     Step 2 — 还原指数权重 (un-normalized):
429 //         w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
430 //     Step 3 — 归一化为混合比例:
431 //         alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
432 //         满足 alpha + beta = 1
433 //     Step 4 — 逐元素加权合并:
434 //         O = alpha * O_1 + beta * O_2
435 //   - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
436 //     lse[head_idx][token_idx])
437 //   - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
438 //     展平为 [T0H0, T0H1, ..., T1H0, ...])
439 //   - -inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score
440 //     为 -inf) 的 LSE 可能为 +inf, 替换后 exp(-inf - L_max) = 0,

```



```

441 // 该段权重退化为 0
442 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
443 //   pack load → FMA → pack store 一条流水线
444 // - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
445 //
446 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
447 //   pack_size = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
448 //   threads_per_head = head_size / 4 = 2
449 //   total_threads = num_tokens * num_heads * threads_per_head = 8
450 //
451 //   global_idx:      0      1      2      3      4      5      6      7
452 //   token_head_idx:  0      0      1      1      2      2      3      3
453 //   (第几个 (token,head) 对, 行优先)
454 //   pack_idx:        0      1      0      1      0      1      0      1
455 //   (该 (token,head) 对内第几个 pack)
456 //   token_idx:       0      0      0      0      1      1      1      1
457 //   (token_head_idx / num_heads, token 变化慢)
458 //   head_idx:        0      0      1      1      0      0      1      1
459 //   (token_head_idx % num_heads, head 变化快)
460 //   pack_offset:     0      4      0      4      0      4      0      4
461 //   (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
462 //   head_offset:     0      0      8      8      16     16     24     24
463 //   (token_idx * num_heads * head_size + head_idx * head_size,
464 //    该 (token,head) 对在 output 展平数组中的起始偏移)
465 //
466 // Grid: ((total_threads + 127) / 128, 1, 1)
467 // Block: (128, 1, 1)
468 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
469 __global__ void merge_attn_states(
470     float *output,           // [num_tokens, num_heads, head_size]
471     const float *prefix_output, // [num_tokens, num_heads, head_size]
472     const float *prefix_lse,   // [num_heads, num_tokens]
473     const float *suffix_output, // [num_tokens, num_heads, head_size]
474     const float *suffix_lse,   // [num_heads, num_tokens]
475     int num_tokens, int num_heads, int head_size) {
476
477     constexpr int NUM_THREADS = 128;
478     constexpr int PACK_SIZE = 16 / sizeof(float); // 4 floats = 128-bit
479     using pack_t = uint4;
480
481     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
482     int threads_per_head = head_size / PACK_SIZE;
483     int total_threads = num_tokens * num_heads * threads_per_head;
484
485     int global_idx = blockIdx.x * NUM_THREADS + threadIdx.x;
486     if (global_idx >= total_threads)
487         return;
488
489     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
490     // token_head_idx: 第几个 (token, head) 对, 行优先展平
491     int token_head_idx = global_idx / threads_per_head;
492     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
493     int pack_idx = global_idx % threads_per_head;
494
495     // token_head_idx 分解为 (token_idx, head_idx)
496     int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
497     int head_idx = token_head_idx % num_heads;   // head 变化快 (内维)
498
499     // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
500     int pack_offset = pack_idx * PACK_SIZE;
501     // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
502     int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
503

```

```

504 // 定位到当前 (token, head) 的输出段起始
505 const float *prefix_head = prefix_output + head_offset;
506 const float *suffix_head = suffix_output + head_offset;
507 float *output_head = output + head_offset;
508
509 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
510 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
511 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
512
513 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
514 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
515 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
516
517 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
518 float max_lse = fmaxf(p_lse, s_lse);
519 p_lse -= max_lse;
520 s_lse -= max_lse;
521
522 // Step 2-3: 指数还原 → 混合比例 alpha, beta
523 float p_se = expf(p_lse);
524 float s_se = expf(s_lse);
525 float out_se = p_se + s_se;
526 float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
527 float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
528
529 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
530 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
531 if (pack_offset < head_size) {
532     pack_t p_pack =
533         reinterpret_cast<const pack_t *>(prefix_head)[pack_offset / PACK_SIZE];
534     pack_t s_pack =
535         reinterpret_cast<const pack_t *>(suffix_head)[pack_offset / PACK_SIZE];
536     pack_t o_pack;
537
538     #pragma unroll
539     for (int i = 0; i < PACK_SIZE; ++i) {
540         float p_val = reinterpret_cast<const float *>(&p_pack)[i];
541         float s_val = reinterpret_cast<const float *>(&s_pack)[i];
542         float o_val = p_val * p_scale + s_val * s_scale;
543         reinterpret_cast<float *>(&o_pack)[i] = o_val;
544     }
545
546     reinterpret_cast<pack_t *>(output_head)[pack_offset / PACK_SIZE] = o_pack;
547 }
548 }
549
550 // =====
551 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
552 // =====
553 // 面试要点:
554 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
555 //     online (1-pass, 增量更新)
556 //   - Online Softmax 是 FlashAttention 的数学基础
557 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
558 //     + variance)
559 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
560
561 // =====
562 // Phase 3a: Softmax — 三级递进 (面试核心考点)
563 // =====
564 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
565 // 回答线索: naive(溢出) → safe → online
566

```

```

567 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
568 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
569 // 问题: x 值很大时 exp(x) 溢出为 inf
570 template <const int NUM_THREADS = 256>
571 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
572 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
573 // source: LeetCUDA/kernels/softmax/softmax.cu
574 __global__ void softmax_per_token(float *x, float *y, int N) {
575     const int tid = threadIdx.x;
576     const int idx = blockIdx.x * blockDim.x + tid;
577
578     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
579     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
580     if (idx < N)
581         y[idx] = exp_val / exp_sum;
582 }
583
584 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
585 // 面试重点: 为什么 Softmax 需要 Safe?
586 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
587 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
588 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
589 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
590 // Grid: (S, 1, 1)
591 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
592 // source: LeetCUDA/kernels/softmax/softmax.cu
593 template <const int NUM_THREADS = 256>
594 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
595     const int tid = threadIdx.x;
596     const int idx = blockIdx.x * blockDim.x + tid;
597
598     // Pass 1: block reduce max — 找最大值
599     float val = (idx < N) ? x[idx] : -FLT_MAX;
600     float max_val = block_reduce_max<NUM_THREADS>(val);
601
602     // Pass 2: exp(x - max) → block reduce sum
603     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
604     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
605
606     if (idx < N)
607         y[idx] = exp_val / exp_sum;
608 }
609
610 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
611 // 面试重点:
612 // 两种使用场景 (公式不同, 但结果等价):
613 // 1) 单元素增量更新 (online update, 处理新元素 x_i):
614 //     m_new = max(m_old, x_i)
615 //     d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
616 // 2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
617 //     m = max(m1, m2)
618 //     d = d1*exp(m1-m) + d2*exp(m2-m)
619 //     当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
620 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
621 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
622 //
623 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
624 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
625 // (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
626 // (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2
627 //
628 // 合并公式 (对称, 不依赖"新/旧"概念):
629 // m = max(m1, m2)

```

```

630 // d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
631 //
632 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
633 struct __align__(8) MD {
634     float m; // running max
635     float d; // running denominator (sum of exp(x - max))
636 };
637
638 template <const int kWarpSize = WARP_SIZE>
639 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
640     #pragma unroll
641     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
642         MD md2;
643         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpSize);
644         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpSize);
645
646         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
647         MD s_m = (md1.m > md2.m) ? md2 : md1;
648
649         // b_m.d 无需 rescale (其基准 max 已是全局 max) ,
650         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
651         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
652         md1.m = b_m.m;
653     }
654     return md1;
655 }
656
657 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
658 // Grid: (S, 1, 1)
659 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
660 // source: LeetCUDA/kernels/softmax/softmax.cu
661 template <const int NUM_THREADS = 256>
662 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
663     int tid = threadIdx.x;
664     int idx = blockIdx.x * NUM_THREADS + threadIdx.x;
665     const int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
666     int warp = tid / WARP_SIZE;
667     int lane = tid % WARP_SIZE;
668
669     // 初始化: 每个线程持有一个 (max, denom) 对
670     MD md;
671     md.m = idx < N ? x[idx] : -FLT_MAX;
672     md.d = idx < N ? 1.0f : 0.0f;
673
674     // Block reduce: warp_reduce_md 在归约中自动更新 m 和 d
675     __shared__ MD shared[NUM_WARPS];
676     md = warp_reduce_md<WARP_SIZE>(md);
677
678     if (lane == 0)
679         shared[warp] = md;
680     __syncthreads();
681
682     // 第二级归约: warp0 收集各 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
683     // 只有 tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
684     if (tid < WARP_SIZE) {
685         md = tid < NUM_WARPS ? shared[tid] : MD{-FLT_MAX, 0.0f};
686         md = warp_reduce_md<NUM_WARPS>(md);
687         if (tid == 0) {
688             shared[0] = md;
689         }
690     }
691     __syncthreads();
692

```

```

693 // 用全局 max 和 denom 做最终 softmax
694 md = shared[0];
695 float d_inv = __fdivdef(1.0f, md.d);
696 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
697 if (idx < N) {
698     y[idx] = __expf(x[idx] - md.m) * d_inv;
699 }
700 }
701
702 // =====
703 // Phase 3b: RMS Normalization (1-pass reduce)
704 // =====
705 // 面试要点:
706 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
707 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
708 //   - Llama 系列使用 RMS Norm
709 //   - grid(N, K/K), block(K): 一行一个 block
710 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
711 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
712 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
713 template <const int NUM_THREADS = 128>
714 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
715     int tid = threadIdx.x;
716     int idx = blockIdx.x * blockDim.x + threadIdx.x;
717     const float epsilon = 1e-5f;
718
719     __shared__ float s_variance;
720     float value = (idx < N * K) ? x[idx] : 0.0f;
721     float variance = value * value;
722     variance = block_reduce_sum<NUM_THREADS>(variance);
723     if (tid == 0)
724         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
725     __syncthreads();
726     if (idx < N * K)
727         y[idx] = (value * s_variance) * g;
728 }
729
730 // RMS Norm + float4
731 // Grid: (N, 1, 1)
732 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
733 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
734 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
735 template <const int NUM_THREADS = 128 / 4>
736 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
737     int tid = threadIdx.x;
738     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
739     const float epsilon = 1e-5f;
740
741     __shared__ float s_variance;
742     float4 reg_x = FLOAT4(x[idx]);
743     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
744                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
745                               : 0.0f;
746     variance = block_reduce_sum<NUM_THREADS>(variance);
747     if (tid == 0)
748         s_variance = rsqrtf(variance / (float)K + epsilon);
749     __syncthreads();
750     float4 reg_y;
751     reg_y.x = reg_x.x * s_variance * g;
752     reg_y.y = reg_x.y * s_variance * g;
753     reg_y.z = reg_x.z * s_variance * g;
754     reg_y.w = reg_x.w * s_variance * g;
755     if (idx < N * K)

```

```

756     FLOAT4(y[idx]) = reg_y;
757 }
758
759 // =====
760 // Phase 3c: Layer Normalization (2-pass reduce)
761 // =====
762 // 面试要点:
763 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
764 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
765 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
766 // Grid: (N, 1, 1), 一行一个 block
767 // Block: (128, 1, 1), NUM_THREADS=K=128
768 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
769 template <const int NUM_THREADS = 128>
770 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
771     int tid = threadIdx.x;
772     int idx = blockIdx.x * blockDim.x + threadIdx.x;
773     const float epsilon = 1e-5f;
774
775     __shared__ float s_mean;
776     __shared__ float s_variance;
777     float value = (idx < N * K) ? x[idx] : 0.0f;
778
779     // Pass 1: compute mean
780     float sum = block_reduce_sum<NUM_THREADS>(value);
781     if (tid == 0)
782         s_mean = sum / (float)K;
783     __syncthreads(); // 必须等待 s_mean 对所有线程可见
784
785     // Pass 2: compute variance = (x - mean)2
786     float variance = (value - s_mean) * (value - s_mean);
787     variance = block_reduce_sum<NUM_THREADS>(variance);
788     if (tid == 0)
789         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
790     __syncthreads(); // 必须等待 s_variance 对所有线程可见
791
792     if (idx < N * K)
793         y[idx] = ((value - s_mean) * s_variance) * g + b;
794 }
795
796 // Layer Norm + float4
797 // Grid: (N, 1, 1)
798 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
799 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
800 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
801 template <const int NUM_THREADS = 128 / 4>
802 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
803     int tid = threadIdx.x;
804     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
805     const float epsilon = 1e-5f;
806
807     __shared__ float s_mean;
808     __shared__ float s_variance;
809     float4 reg_x = FLOAT4(x[idx]);
810     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
811
812     float sum = block_reduce_sum<NUM_THREADS>(value);
813     if (tid == 0)
814         s_mean = sum / (float)K;
815     __syncthreads();
816
817     float4 reg_x_hat;
818     reg_x_hat.x = reg_x.x - s_mean;

```

```

819 reg_x_hat.y = reg_x.y - s_mean;
820 reg_x_hat.z = reg_x.z - s_mean;
821 reg_x_hat.w = reg_x.w - s_mean;
822 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
823                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
824 variance = block_reduce_sum<NUM_THREADS>(variance);
825 if (tid == 0)
826     s_variance = rsqrtf(variance / (float)K + epsilon);
827 __syncthreads();
828
829 float4 reg_y;
830 reg_y.x = reg_x_hat.x * s_variance * g + b;
831 reg_y.y = reg_x_hat.y * s_variance * g + b;
832 reg_y.z = reg_x_hat.z * s_variance * g + b;
833 reg_y.w = reg_x_hat.w * s_variance * g + b;
834 if (idx < N * K)
835     FLOAT4(y[idx]) = reg_y;
836 }
837
838 // =====
839 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
840 // =====
841 // 面试要点:
842 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
843 //     [x1']  [cos(θ) -sin(θ)] [x1]
844 //     [x2']  [sin(θ)  cos(θ)] [x2]
845 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
846 //   - token_pos = idx / N: token 在序列中的位置
847 //   - token_idx = idx % N: token 内的维度对索引
848 //   - 输入 [seq_len, hidden_size], 输出同形状
849
850 // Grid: ((seq_len * N + 255) / 256, 1, 1)
851 // Block: (256, 1, 1)
852 // source: LeetCUDA/kernels/rope/rope.cu
853 __global__ void rope(float *x, float *out, int seq_len, int N) {
854     int idx = blockIdx.x * blockDim.x + threadIdx.x;
855     float x1 = x[idx * 2];
856     float x2 = x[idx * 2 + 1];
857     int token_pos = idx / N; // 序列位置
858     int token_idx = idx % N; // 维度对索引
859
860     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
861     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
862     float sin_v = sinf(token_pos * exp_v);
863     float cos_v = cosf(token_pos * exp_v);
864
865     // 2D 旋转
866     float out1 = x1 * cos_v - x2 * sin_v;
867     float out2 = x1 * sin_v + x2 * cos_v;
868     out[idx * 2] = out1;
869     out[idx * 2 + 1] = out2;
870 }
871
872 // =====
873 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
874 // =====
875 // 面试要点 (Bank Conflict 专题):
876 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
877 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
878 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
879 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
880 //
881 // 转置的四步演进:

```



```

882 // naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
883 // shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
884 // BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
885 // merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
886 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
887
888 // 每个线程处理 1 个元素, block(16,16)
889 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
890 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
891 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
892 // Block: (16, 16, 1)
893 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
894 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
895     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
896     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
897     if (r < row && c < col) {
898         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
899         y[c * row + r] = x[r * col + c];
900     }
901 }
902
903 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
904 // Block: (16, 16, 1)
905 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
906 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
907 __global__ void mat_transpose_padded(
908     float *x, float *y, const int row, const int col) {
909     const int tx = threadIdx.x, ty = threadIdx.y;
910
911     constexpr int TILE = 16;
912     constexpr int PAD = 1;
913     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
914     __shared__ float tile[TILE * 4][TILE + PAD];
915
916     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
917     const int x_c = blockIdx.x * TILE + tx;
918     const int x_r = blockIdx.y * TILE + ty;
919
920     if (x_r * 4 < row && x_c < col) {
921         // Step 1: 4 rows × 1 col → smem (row-major);
922 #pragma unroll
923         for (int i = 0; i < 4; ++i) {
924             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
925         }
926         __syncthreads();
927
928         // Step 2: Transposed read from smem
929         float4 reg_trans;
930         reg_trans.x = tile[tx * 4 + 0][ty];
931         reg_trans.y = tile[tx * 4 + 1][ty];
932         reg_trans.z = tile[tx * 4 + 2][ty];
933         reg_trans.w = tile[tx * 4 + 3][ty];
934
935         // y 空间坐标: y_r = x 的列, y_c = x 的行
936         const int y_r = blockIdx.x * TILE + ty; // = x col
937         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
938
939         // Coalesced write: y[y_r][y_c .. y_c+3]
940         FLOAT4(y[y_r * row + y_c]) = reg_trans;
941     }
942 }
943
944 // =====

```



```

945 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
946 // =====
947 // 面试要点:
948 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
949 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
950 //   - 不同 K 值对应不同分块策略:
951 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
952 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
953 //       K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
954 //
955 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
956
957 // ---- SGEMV K32: 基础 warp-per-row ----
958 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
959 // grid(M/4), 每个 warp 负责一行
960 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
961 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
962 // Block: (32, 4, 1), 每行 1 warp
963 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
964 // source: LeetCUDA/kernels/sgemv/sgemv.cu
965 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
966     int tx = threadIdx.x; // 0~31
967     int ty = threadIdx.y; // 0~3
968     int lane = tx % WARP_SIZE; // 0~31
969     int m = blockIdx.x * blockDim.y + ty; // 全局行号
970     if (m < M) {
971         float sum = 0.0f;
972         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
973         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
974         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
975 #pragma unroll
976         for (int w = 0; w < NUM_ITERS; ++w) {
977             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
978             int k = w * WARP_SIZE + lane;
979             sum += a[m * K + k] * x[k];
980         }
981         sum = warp_reduce_sum<WARP_SIZE>(sum);
982         // 每个 warp 处理一行, lane 0 写回结果
983         if (lane == 0)
984             y[m] = sum;
985     }
986 }
987
988 // ---- SGEMV K128: float4 向量化 ----
989 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
990 // Grid: ((M + 3) / 4, 1, 1)
991 // Block: (32, 4, 1)
992 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
993 // source: LeetCUDA/kernels/sgemv/sgemv.cu
994 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
995     int tx = threadIdx.x; // 0~31
996     int ty = threadIdx.y; // 0~3
997     int lane = tx % WARP_SIZE; // 0~31
998     int m = blockDim.y * blockIdx.x + ty;
999
1000     if (m < M) {
1001         float sum = 0.0f;
1002         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1003         const int NUM_ITERS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
1004 #pragma unroll
1005         for (int w = 0; w < NUM_ITERS; ++w) {
1006             int k = (w * WARP_SIZE + lane) * 4;
1007             float4 reg_x = FLOAT4(x[k]);

```

```

1008     float4 reg_a = FLOAT4(a[m * K + k]);
1009     sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1010            reg_a.w * reg_x.w);
1011 }
1012 sum = warp_reduce_sum<WARP_SIZE>(sum);
1013 if (lane == 0)
1014     y[m] = sum;
1015 }
1016 }
1017
1018 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
1019 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1020 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1021 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1022 // Block: (32, 4, 1)
1023 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
1024 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1025 template <const int ROW_PER_WARP = 2>
1026 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1027     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
1028     int tx = threadIdx.x;
1029     int ty = threadIdx.y;
1030     int lane = tx % WARP_SIZE;
1031     int k = lane % K_WARP_SIZE; // 0~15
1032     int m = (blockDim.y * blockIdx.x + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
1033     if (m < M) {
1034         float sum = A[m * K + k] * x[k];
1035         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1036         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
1037         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1038         if (k == 0)
1039             y[m] = sum;
1040     }
1041 }
1042
1043 // =====
1044 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1045 // =====
1046 // 面试要点 (GEMM 优化五层金字塔):
1047 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1048 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1049 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1050 //   load/store 指令数 Level 4 — Tensor Core (MMA
1051 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1052 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1053 //
1054 // 计算密度递进:
1055 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1056 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1057 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1058
1059 // =====
1060 // Phase 7a: SGEMM + HGEMM (非 Tensor Core 路径)
1061 // =====
1062
1063 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1064 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1065 // C = A × B, C[M, N] = A[M, K] × B[K, N]
1066 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1067 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1068 // Block: (32, 32, 1), 1024 线程
1069 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1070 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {

```

```

1071 constexpr int BM = 32; // vec 版: 32x4 = 128
1072 constexpr int BN = 32; // vec 版: 32x4 = 128
1073 constexpr int BK = 32;
1074 __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1075
1076 int bx = blockIdx.x;
1077 int by = blockIdx.y;
1078 int tx = threadIdx.x;
1079 int tid = threadIdx.y * blockDim.x + tx;
1080
1081 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1082 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1083 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1084 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1085 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1086 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1087 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1088
1089 float sum = 0.f; // 遍历完整的K, slice K;
1090 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1091     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1092     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1093     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1094     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1095     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1096     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1097     __syncthreads(); // 确保整个 smem tile 加载完毕
1098 }
1099 #pragma unroll
1100 for (int k = 0; k < BK; ++k) {
1101     int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1102     int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1103     sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1104 }
1105 __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1106 }
1107 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1108 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1109 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1110 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1111 }
1112
1113 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1114 // 在 Level 1 基础上引入两层优化:
1115 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1116 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1117 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1118 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1119 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1120 // 1024 x 16 = 16384 = 128 x 128 ✓
1121 //
1122 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1123 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1124 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1125 //
1126 // 加载映射 (每线程 4 个元素, float4):
1127 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1128 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1129 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1130 //
1131 // △ Bank Conflict 提示 (面试加分点):
1132 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1133 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用

```

```

1134 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1135 //
1136 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1137 // Block: (32, 32, 1), 1024 线程
1138 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1139 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1140 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1141     constexpr int BM = 128;
1142     constexpr int BN = 128;
1143     constexpr int BK = 32;
1144     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1145     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1146
1147     int bx = blockIdx.x;
1148     int by = blockIdx.y;
1149     int tx = threadIdx.x;
1150     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1151
1152     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1153     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1154     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1155     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1156     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1157     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1158
1159     int load_gmem_a_m = by * BM + load_smem_a_m;
1160     int load_gmem_b_n = bx * BN + load_smem_b_n;
1161
1162     // 4x4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
1163     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1164     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1165
1166     float sum[4][4] = {0.f};
1167     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1168         int load_gmem_a_k = bk * BK + load_smem_a_k;
1169         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1170         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1171         int load_gmem_b_k = bk * BK + load_smem_b_k;
1172         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1173         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1174         __syncthreads();
1175
1176     #pragma unroll
1177     for (int k = 0; k < BK; ++k) {
1178         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1179         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1180                             s_a[comp_smem_a_m_base + 1][k],
1181                             s_a[comp_smem_a_m_base + 2][k],
1182                             s_a[comp_smem_a_m_base + 3][k]};
1183         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1184                             s_b[k][comp_smem_b_n_base + 1],
1185                             s_b[k][comp_smem_b_n_base + 2],
1186                             s_b[k][comp_smem_b_n_base + 3]};
1187
1188     #pragma unroll
1189     for (int i = 0; i < 4; ++i) {
1190     #pragma unroll
1191         for (int j = 0; j < 4; ++j) {
1192             sum[i][j] += a_vals[i] * b_vals[j];
1193         }
1194     }
1195     __syncthreads();
1196 }

```

```

1197 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1198 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1199 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1200 #pragma unroll
1201 for (int i = 0; i < 4; ++i) {
1202     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1203     float4 reg_c;
1204     reg_c.x = sum[i][0];
1205     reg_c.y = sum[i][1];
1206     reg_c.z = sum[i][2];
1207     reg_c.w = sum[i][3];
1208     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1209 }
1210 }
1211 }
1212
1213 // =====
1214 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1215 // =====
1216 // 面试要点:
1217 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1218 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1219 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1220 // 寄存器)
1221 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1222 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1223 //
1224 // ★ TN 布局详解 (面试高频考点):
1225 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1226 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1227 // N = Normal (列优先, BLAS 原生格式)
1228 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1229 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1230 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1231 // BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1232 // 视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1233 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1234 // T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1235 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1236 //
1237 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1238 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1239 // N = col-major (列优先, BLAS native = normal)
1240 //
1241 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]
1242 // - A: row-major [M, K], B: row-major [K, N]
1243 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1244 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1245 //
1246 // TN 布局: C[MxN] = A[MxK] × B[KxN], B 以 B^T=[NxK] row-major 存储 (A 行优先, B 列优先)
1247 // - A [MxK]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1248 // - B^T [NxK]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1249 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1250 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1251 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1252 //
1253 // WGMMMA (Warp Group MMA, Hopper+):
1254 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1255 // - m64n128k16: 一次处理 64x128x16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1256 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1257 // threads) 做计算
1258 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1259

```

```

1260 // =====
1261 // Phase 7b-1: MMA PTX 宏定义
1262 // =====
1263
1264 // ---- gmem → smem: cp.async ----
1265 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1266 //   - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1267 //   - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1268 //     N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1269 //   - wait_all: 等价于 commit_group + wait_group 0。
1270 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1271 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1272 #define CP_ASYNC_WAIT_GROUP(n) \
1273     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1274
1275 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1276 #define CP_ASYNC_CG(dst, src, bytes) \
1277     asm volatile( \
1278         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1279         "l"(src), "n"(bytes))
1280
1281 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1282 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1283 // 每次加载 8x8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1284 // aligned: 要求 128-bit 对齐, trans: 转置加载
1285 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1286     asm volatile( \
1287         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1288         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1289         : "r"(addr))
1290
1291 #define LDMATRIX_X2(R0, R1, addr) \
1292     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1293         : "=r"(R0), "=r"(R1) \
1294         : "r"(addr))
1295
1296 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1297 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1298 #define LDMATRIX_X2_T(R0, R1, addr) \
1299     asm volatile( \
1300         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1301         : "=r"(R0), "=r"(R1) \
1302         : "r"(addr))
1303
1304 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1305 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1306 // row.col: A 是 row-major, B 是 col-major
1307 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1308 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1309 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1310 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1311     asm volatile( \
1312         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1313         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1314         : "=r"(RD0), "=r"(RD1) \
1315         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1316         "r"(RC1))
1317
1318 // ---- MMA 辅助函数 ----
1319 // div_ceil: 整数除法向上取整
1320 #define HOST_DEVICE_INLINE __device__ __host__ inline
1321 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1322     return (a % b != 0) ? (a / b + 1) : (a / b);

```



```

1323 }
1324
1325 // =====
1326 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1327 // =====
1328 // 面试重点 — Tile Hierarchy:
1329 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1330 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1331 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1332 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1333 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1334 //
1335 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1336 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1337 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1338 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1339 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1340 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1341 //
1342 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1343 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1344 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1345 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1346 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1347 //
1348 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1349 // Block: (256, 1, 1), 8 warps
1350 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1351 // =====
1352 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1353           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1354           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1355           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1356 __global__ void __launch_bounds__(256)
1357   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1358   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1359   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1360   const int by = blockIdx.y;
1361   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1362   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1363   constexpr int BK = MMA_K; // 16
1364
1365   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1366   // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1367   // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1368   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1369   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1370   extern __shared__ half smem[];
1371   half *s_a = smem;
1372   half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1373   constexpr int s_a_stage_offset = BM * BK; // 128*16
1374   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major
1375   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1376   const int warp_id = tid / WARP_SIZE; // 0~7
1377   const int lane_id = tid % WARP_SIZE; // 0~31
1378   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1379   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1380
1381   // 线程到 global memory 的映射 (用于加载 A 和 B)
1382   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1383   int load_smem_a_m = tid / 2; // 0~127
1384   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1385   int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)

```

```

1386 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1387 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1388 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1389 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1390     return;
1391
1392 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1393 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1394 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是VAL Tile的作用。
1395 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1396
1397 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1398 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1399 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1400
1401 // 预加载前 (K_STAGE-1) 个 stage
1402 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1403 #pragma unroll
1404 for (int k = 0; k < (K_STAGE - 1); ++k) {
1405     int load_gmem_a_k = k * BK + load_smem_a_k;
1406     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1407     int load_gmem_b_k = k * BK + load_smem_b_k;
1408     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1409     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1410
1411     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1412         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1413     );
1414     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1415
1416     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1417         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1418     );
1419     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1420
1421     CP_ASYNC_COMMIT_GROUP();
1422 }
1423
1424 const int NUM_K_TILES = div_ceil(K, BK);
1425 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1426 __syncthreads();
1427
1428 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1429 #pragma unroll
1430 for (int k = 0; k < NUM_K_TILES; ++k) {
1431     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1432     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1433
1434     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1435     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k (row-major, 内维连续的是 K)
1436     if (k + K_STAGE - 1 < NUM_K_TILES) {
1437         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1438         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1439         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1440         // B^T: row-major [n][k], 内维连续的是 K △
1441         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1442
1443         uint32_t load_smem_a_ptr =
1444             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1445                 load_smem_a_m * BK + load_smem_a_k) *
1446                 sizeof(half));
1447         CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1448     }
1449 }

```



```

1449     uint32_t load_smem_b_ptr =
1450         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1451             load_smem_b_n * BK + load_smem_b_k) *
1452             sizeof(half));
1453     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1454     CP_ASYNC.COMMIT_GROUP();
1455 }
1456
1457 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1458 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1459 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1460 uint32_t RA[VAL_TILE_M][4];
1461 uint32_t RB[VAL_TILE_N][2];
1462
1463 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1464 #pragma unroll
1465 for (int i = 0; i < VAL_TILE_M; ++i) {
1466     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1467     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1468     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1469     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1470     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1471     uint32_t lane_smem_a_ptr =
1472         (smem_a_base_ptr +
1473             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1474             sizeof(half));
1475     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1476 }
1477
1478 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1479 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1480 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1481 #pragma unroll
1482 for (int j = 0; j < VAL_TILE_N; ++j) {
1483     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1484     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1485     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1486     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1487     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1488     uint32_t lane_smem_b_ptr =
1489         (smem_b_base_ptr +
1490             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1491             sizeof(half));
1492     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1493 }
1494
1495 // MMA compute: 发射 VAL_TILE_M × VAL_TILE_N 条 MMA 指令
1496 #pragma unroll
1497 for (int i = 0; i < VAL_TILE_M; ++i) {
1498     #pragma unroll
1499     for (int j = 0; j < VAL_TILE_N; ++j) {
1500         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1501             RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1502             RB[j][0], RB[j][1], // B fragment
1503             RC[i][j][0], RC[i][j][1]);
1504     }
1505 }
1506
1507 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1508 if (k + K_STAGE - 1 < NUM_K_TILES) {
1509     CP_ASYNC.WAIT_GROUP(K_STAGE - 2);
1510 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1511     CP_ASYNC.WAIT_GROUP(0);

```

```

1512 }
1513 __syncthreads();
1514 }
1515
1516 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1517 {
1518     for (int i = 0; i < VAL_TILE_M; ++i) {
1519         uint32_t RC0[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1520         uint32_t RC1[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1521         // =====
1522         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1523         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1524         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1525         //
1526         //      cols: c0-1    c2-3    c4-5    c6-7
1527         //  r0:  T0{c0,c1}   T1       T2       T3       |
1528         //  r1:   T4         T5       T6       T7       |
1529         //  r2:  T8      →   T9      →   T10     →   T11     | RC[0]
1530         //  ...              |
1531         //  r7:  T28         T29       T30       T31       |
1532         //  r8:  T0{c2,c3}   T1       T2       T3       |
1533         //  r9:   T4         T5       T6       T7       |
1534         //  r10: T8      →   T9      →   T10     →   T11     | RC[1]
1535         //  ...              |
1536         //  r15: T28         T29       T30       T31       |
1537         //
1538         // Within a 4-lane group (e.g. T0-T3 for row 0):
1539         //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1540         //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1541         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1542         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1543         // =====
1544 #pragma unroll
1545         for (int j = 0; j < VAL_TILE_N; ++j) {
1546             RC0[j][0] = RC[i][j][0];
1547             RC1[j][0] = RC[i][j][1];
1548             RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1549             RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1550             RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1551             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1552             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1553             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1554         }
1555         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1556         // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1557         //
1558         // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1559         // |-----|-----|-----|-----|
1560         // | 0~3   | 0       | row 0     | row 8     |
1561         // | 4~7   | 1       | row 1     | row 9     |
1562         // | 8~11  | 2       | row 2     | row 10    |
1563         // | 12~15 | 3       | row 3     | row 11    |
1564         // | 16~19 | 4       | row 4     | row 12    |
1565         // | 20~23 | 5       | row 5     | row 13    |
1566         // | 24~27 | 6       | row 6     | row 14    |
1567         // | 28~31 | 7       | row 7     | row 15    |
1568         //
1569         if (lane_id % 4 == 0) {
1570             // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1571             int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1572             int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1573 #pragma unroll
1574             for (int j = 0; j < VAL_TILE_N; ++j) {

```

```

1575 // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1576 int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1577 int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1578 int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1579 int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1580 // 128-bit store: 一次写入 8 个 half
1581 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1582     *reinterpret_cast<float4*>(&RC0[j][0]);
1583 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1584     *reinterpret_cast<float4*>(&RC1[j][0]);
1585 }
1586 }
1587 }
1588 }
1589 }
1590
1591 // =====
1592 // Phase 7c: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
1593 // =====
1594 // 面试要点 (WGMMMA vs MMA 对比):
1595 // - MMA: warp 级 (32 threads), 同步执行
1596 // - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
1597 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
1598 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1599 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
1600 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1601
1602 // ---- WGMMMA 辅助函数 ----
1603 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7):
1604 // - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
1605 //   (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行)。
1606 // - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
1607 //   N=0 → 等所有 group 完成。* 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),
1608 //   都是 "wait until only N or fewer groups are pending"。
1609 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
1610 #define WGMMMA_COMMIT_GROUP() \
1611     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
1612 #define WGMMMA_WAIT_GROUP(n) \
1613     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :::"n"(n) : "memory")
1614
1615 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
1616 // 编码规则 (PTX ISA §9.7.15.5.1.2.2):
1617 //   encoded = (x & 0x3FFFF) >> 4
1618 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
1619 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
1620 // 可省略以扩大可表示范围。
1621 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
1622 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
1623
1624 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
1625 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
1626 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
1627 //
1628 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor):
1629 // -----
1630 // | 位域          | 范围      | 大小 | 含义
1631 // |-----|-----|-----|-----|
1632 // | start_address  | [0, 14)   | 14   | smem 基址编码
1633 // | (unused)       | [14, 16)  | 2    | -
1634 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
1635 // | (unused)       | [30, 32)  | 2    | -
1636 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
1637 // | (unused)       | [46, 49)  | 3    | -

```

```

1638 // | base_offset      | [49, 52) | 3 | swizzle pattern 偏移
1639 // | (unused)         | [52, 62) | 10 | -
1640 // | layout_type      | [62, 64) | 2 | swizzle 模式
1641 // -----
1642 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
1643 //
1644 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
1645 // - 128B swizzle atom 在 128-bit 单位下为 8×8
1646 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
1647 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
1648 // - 这是 stride_byte_offset=1024 的来源
1649 //
1650 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
1651 //   此处填 16 仅为占位, 编码后为 1。
1652 //
1653 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
1654 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
1655 //
1656 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
1657 device inline uint64_t make_smem_desc(half *ptr) {
1658     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
1659     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
1660     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
1661
1662     // 从全零开始: 所有位域初始化为 0。
1663     uint64_t desc = 0x0000000000000000;
1664
1665     // — bits [0, 14): start_address —
1666     // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
1667     // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
1668     // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
1669     desc |= SMEM_DESC_ENCODE(addr);
1670
1671     // — bits [16, 30): leading_byte_offset —
1672     // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
1673     // << 16 将编码值放到 bits [16, 30)。
1674     // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
1675     // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
1676     // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
1677     //   对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
1678     //   对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
1679     desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
1680
1681     // — bits [32, 46): stride_byte_offset —
1682     // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
1683     // << 32 将编码值放到 bits [32, 46)。
1684     // K-Major 下定义为“从首 8 行到次 8 行的字节偏移” (PTX ISA §9.7.15.5.1.2.1.2) 。
1685     // 1024 的来源 (K-Major + 128B swizzle + half) :
1686     //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
1687     //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
1688     // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
1689     desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
1690
1691     // — bits [62, 64): layout_type (swizzle mode) —
1692     // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
1693     // 1 = 128B swizzle mode。
1694     // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
1695     // 128B swizzle 将 smem 中连续的 row 按 128B 粒度进行 XOR 重映射,
1696     // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
1697     desc |= 1llu << 62;
1698
1699     return desc;
1700 }

```

```

1701
1702 // ---- WGMMA PTX 指令宏 ----
1703 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1704 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算 D[64×128] += A[64×16] × B[16×128]。
1705 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
1706 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
1707 // 每线程 64 half = 32 uint32。
1708 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
1709 // Scaled: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
1710 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
1711 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
1712 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
1713 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, Scaled, ScaleA, ScaleB, TransA, \
1714                                     TransB) \
1715 { \
1716     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
1717     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
1718     asm volatile( \
1719         "{\n" \
1720         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
1721         "{%0,  %1,  %2,  %3,  %4,  %5,  %6,  %7,  " \
1722         " %8,  %9,  %10, %11, %12, %13, %14, %15, " \
1723         " %16, %17, %18, %19, %20, %21, %22, %23, " \
1724         " %24, %25, %26, %27, %28, %29, %30, %31}, " \
1725         " %32, " \
1726         " %33, " \
1727         " %34, %35, %36, %37, %38;\n" \
1728         "}\n" \
1729         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
1730           "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
1731           "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
1732           "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
1733           "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
1734           "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
1735           "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
1736           "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
1737         : "l"(desc_a), "l"(desc_b), "n"(int32_t(Scaled)), \
1738           "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
1739           "n"(int32_t(TransB))); \
1740     }
1741
1742 // ---- TMA Shared Memory Layout ----
1743 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
1744 //
1745 // 每个 stage 包含两块 smem:
1746 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
1747 //   B tile: [BK, BN] row-major → BN×BK 个 half, 地址连续
1748 //
1749 // ★ 关键区别 — WGMMA 的 B 布局 vs MMA(TN) 的 B^T 布局:
1750 //   MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 解出 col-major B fragment。
1751 //   WGMMA:    smem 存 B [BK, BN] row-major (即 N 维连续, K 维步幅=BN个half)。
1752 //             WGMMA 指令的 imm-trans-b=0 指示硬件按 K-major 读 B,
1753 //             128B swizzle + make_smem_desc 的 stride_byte_offset 将这些地址
1754 //             重新映射, 使硬件能从 [BK, BN] row-major 中正确按 K-major 顺序取值。
1755 //   简言之: swizzle 桥接了 "row-major 存储" 和 "K-major 读取" 的差异。
1756 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
1757     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
1758     // B: K-major 布局, 供 WGMMA imm-trans-b=0 直接读取, [BK, BN]。
1759     alignas(128) half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN]
1760 };
1761
1762 // ---- WGMMA Kernel: Warp Specialization + TMA ----
1763 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):

```

```

1764 //
1765 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
1766 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
1767 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
1768 //
1769 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
1770 //   将 A/B tile 从 HBM 搬到 shared memory。
1771 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
1772 //
1773 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
1774 //   - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
1775 //   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
1776 //
1777 // 本节重点理解:
1778 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
1779 //      即可搬运整个 2D tile, 无需所有线程参与。
1780 //   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
1781 //   3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
1782 //      Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
1783 //
1784 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
1785 //   WGMMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
1786 //   K Tile:           BK=64, 每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
1787 //   M Tile:           BM=128, 每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
1788 //   N Tile:           BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块
1789 //   Block Tile:       C[128,128] = A[128,64] × B[64,128]
1790 //   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
1791 //   Producer(WG0):    128 threads (仅 thread 0 做 TMA 提交)
1792 //   Consumer(WG1):    128 threads = 4 warps (全部参与 WGMMMA 和写回)
1793 //
1794 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
1795 // 的延迟被计算完全掩盖。
1796 //
1797 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1798 //   - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
1799 // Block: (256, 1, 1), 2 warpgroups
1800 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
1801 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
1802           const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
1803           const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
1804           const bool BLOCK_SWIZZLE = false>
1805 __global__ void __launch_bounds__(NUM_THREADS)
1806   hgemm_wgmma_stages_tn(
1807     int M, int N, int K, half *C,
1808     const CUtensorMap *__restrict__ tensorMapA,
1809     const CUtensorMap *__restrict__ tensorMapB) {
1810   // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
1811   // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
1812   // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
1813   // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
1814   // 不展开宿主侧 create_tensor_map 细节。
1815
1816   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1817   // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
1818   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1819   const int by = blockIdx.y;
1820   // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
1821   // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
1822   constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
1823   // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
1824   // 当只有一个 consumer 时, 它负责全部 BM=128 行
1825   constexpr int B_WG_M = BM / num_consumers; // 128
1826

```



```

1827 // 边界检查: 确保当前 block 不超出 M/N 范围
1828 if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1829     return;
1830
1831 // ---- Shared Memory 分配 ----
1832 // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
1833 // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
1834 extern __shared__ __align__(128) uint8_t smem_AB[];
1835 WgmmaSMem<BM, BN, BK, K_STAGE> &s =
1836     *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE> *>(smem_AB);
1837 half *s_a = s.A;
1838 half *s_b = s.B;
1839
1840 // ---- cuda::barrier 初始化 ----
1841 //
1842 // ★ Barrier 机制速查 (arrive / wait 核心语义):
1843 //
1844 // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
1845 //
1846 //   init(bar, N): 设置 arrive_count = N。
1847 //       含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
1848 //
1849 //   arrive(): 线程声明"我已到达此 barrier 点"。
1850 //       - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
1851 //       - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
1852 //
1853 //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
1854 //       - 即: 等待 phase 翻转。
1855 //       - Phase 翻转条件: (1) arrive 次数达到 arrive_count
1856 //                           (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
1857 //
1858 //   典型用法: b.wait(b.arrive())
1859 //       - arrive() 注册自己到达, 拿到 token (当前 phase = P)
1860 //       - wait(token) 阻塞直到 phase 翻转 (P → P+1)
1861 //       - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
1862 //       - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
1863 //
1864 // ★ 本 kernel 的 Pipeline 同步协议 (K_STAGE=3, arrive_count=129):
1865 //
1866 //   Producer (1 thread)           Consumer (128 threads)
1867 //   ────────────                   ────────────
1868 //   C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
1869 //   ┌ for each k_tile: ─┐           ┌ for each k_tile: ─┐
1870 //   │ P1: arrive+wait(empty[q]) │   │ C1: arrive+wait(full[q]) │
1871 //   │   ↓ 等 stage q 被消费完   │   │   ↓ 等 TMA 数据就绪   │
1872 //   │ P2: TMA(A[q]) + TMA(B[q]) │   │ C2: WGMMMA 计算   │
1873 //   │   ↓ 异步拷贝             │   │ C3: wait WGMMMA 完成 │
1874 //   │ P3: arrive_tx(full[q])    │   │ C4: arrive(empty[q]) │
1875 //   │   ↑ 通知: stage q 数据就绪 │   │   ↑ 通知: stage q 已消费完 │
1876 //   └──────────────────┘           └──────────────────┘
1877 //
1878 // 关键不变式 (invariant):
1879 //   - Producer 不会覆盖 Consumer 正在读的 stage
1880 //   - Consumer 不会读 Producer 还没写完的 stage
1881 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
1882 //
1883 // 每个 stage 有两个 barrier:
1884 //   full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
1885 //   empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
1886 //
1887 // arrive_count = 128 (consumer) + 1 (producer) = 129:
1888 //   每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
1889 #pragma nv_diag_suppress static_var_with_dynamic_init

```



```

1890 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1891 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
1892 #pragma nv_diag_default static_var_with_dynamic_init
1893
1894 // K 方向总 tile 数。要求 K 能被 BK 整除，否则尾 tile 被丢弃。
1895 const int num_blocks_k = K / BK;
1896 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1897 const int tid = threadIdx.x % 128;    // 0~127 within warpgroup
1898
1899 // 初始化 barriers (仅 thread 0 执行)
1900 if (threadIdx.x == 0) {
1901     for (int i = 0; i < K_STAGE; ++i) {
1902         // arrive_count = 129: 128 consumer + 1 producer
1903         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内)，
1904         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
1905         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
1906         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
1907         init(&empty[i], num_consumers * 128 + 1); // same
1908     }
1909     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
1910     // 对 async proxy (TMA/WGMMMA) 可见。
1911     cuda::device::experimental::fence_proxy_async_shared_cta();
1912 }
1913 __syncthreads();
1914
1915 // =====
1916 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
1917 // =====
1918 //
1919 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工，不是先后顺序：
1920 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支，做 Producer。
1921 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支，做 Consumer。
1922 //
1923 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp，两者**同时**推进：
1924 //   Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
1925 //   Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
1926 //
1927 // 两个 warpgroups 各自独立迭代 K-tiles，通过 full[] / empty[] barrier 同步：
1928 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞，等 Consumer 消费完
1929 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞，等 TMA 搬运完
1930 //
1931 // 这是生产者-消费者模型的 GPU 实现：不需要共享循环变量，barrier 的 phase
1932 // 交替天然保证了流水线串行化 (at most K_STAGE-1 steps ahead)。
1933 // =====
1934 // Producer Warpgroup (WG0, threadIdx.x 0~127)
1935 // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
1936 // 只有 tid==0 执行实际拷贝提交，其余 127 个线程空闲。
1937 // =====
1938 if (wg_idx == 0) {
1939     if (tid == 0) {
1940         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
1941         int qidx = 0;
1942         for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
1943             if (qidx == K_STAGE)
1944                 qidx = 0;
1945
1946             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
1947             //
1948             // 模式 empty[qidx].wait(empty[qidx].arrive()):
1949             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达，
1950             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
1951             //     128 次 arrive (来自上一轮迭代或 C0 初始化)，则加上这次共 129 次
1952             //     → phase 立即翻转。

```

```

1953 // - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
1954 // 由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
1955 // wait 看到 phase 已变 → 立即返回(首轮不阻塞)。
1956 //
1957 // 语义: Producer 说"我准备好写 stage qidx 了, Consumer 用完没有?"
1958 // 如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
1959 // 如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
1960 empty[qidx].wait(empty[qidx].arrive());
1961
1962 // Step P2: 提交 TMA 2D 拷贝指令
1963 // cp_async_bulk_tensor_2d_global_to_shared:
1964 // 参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
1965 // 参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
1966 // 参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
1967 // 参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
1968 //
1969 // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
1970 // 无需线程逐元素搬运, 零寄存器开销。
1971 // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1972 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1973     &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1974     full[qidx]);
1975
1976 // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1977 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1978     &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1979     full[qidx]);
1980
1981 // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
1982 //
1983 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
1984 // - 在 bar 上注册 1 次到达 (arrive_count_update=1)
1985 // - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
1986 // - Phase 翻转条件:
1987 // (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
1988 // (b) 所有声明的 async 字节已写入 smem
1989 // 两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
1990 // - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
1991 [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
1992     full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
1993 }
1994 }
1995 }
1996 // =====
1997 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
1998 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
1999 // 所有 128 个线程 (4 warps) 全部参与。
2000 // =====
2001 else {
2002     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
2003     //
2004     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
2005     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[qidx].wait()
2006     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
2007     //
2008     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
2009     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
2010     for (int i = 0; i < K_STAGE; ++i) {
2011         [[maybe_unused]] auto token = empty[i].arrive();
2012     }
2013
2014     // 累加器寄存器声明
2015     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:

```

```

2016 // - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
2017 // - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
2018 // - d[*][g][*]: N 方向第 g 组 16 列
2019 // - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
2020 // 每个线程总共 2 * 8 * 4 = 64 uint32 = 128 half
2021 // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
2022 uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
2023
2024 int qidx = 0;
2025 // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
2026 for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
2027     if (qidx == K_STAGE)
2028         qidx = 0;
2029
2030     // Step C1: 等待 TMA 数据就绪 (full 信号)
2031     //
2032     // 模式 full[qidx].wait(full[qidx].arrive()):
2033     // - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
2034     // - 当前 phase 累积: 128/129, phase 尚未翻转。
2035     // - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
2036     //   贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
2037     //
2038     // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有?"
2039     full[qidx].wait(full[qidx].arrive());
2040
2041     // Step C2: 发射 WGMMMA 指令序列
2042     //
2043     // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
2044     // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
2045     //
2046     // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
2047     // - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
2048     // - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
2049     // - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
2050     //
2051     // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
2052     // D[64x128] += A[64x16] × B[16x128]
2053     // A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
2054     // Scaled=1: 累加。K 维有 BK/WGMMMA_K=4 次迭代, 每次都 Scaled=1。
2055     WGMMMA_FENCE();
2056
2057     // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
2058     // 每个 atom 处理 64 行 M 方向数据。
2059 #pragma unroll
2060     for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2061         // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
2062         // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
2063         half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
2064
2065         // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
2066         // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
2067 #pragma unroll
2068         for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
2069             // 第 k_it 次 WGMMMA:
2070             // A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
2071             // B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
2072             // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
2073             // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
2074             // Scaled=1: 累加 (K 维迭代需要累积结果)
2075             WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
2076                                     s_b + qidx * BK * BN + k_it * WGMMMA_K,
2077                                     1, // Scaled=1: accumulate (不清零)
2078                                     1, 1, 0, 0);

```

```

2079     }
2080 }
2081
2082 // Step C3: 提交并等待 WGMMMA 完成
2083 //
2084 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
2085 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
2086 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
2087 //
2088 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
2089 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
2090 // * 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
2091 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
2092 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
2093 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
2094 WGMMMA_COMMIT_GROUP();
2095 WGMMMA_WAIT_GROUP(0);
2096
2097 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
2098 //
2099 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
2100 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
2101 //
2102 // 语义: Consumer 说"stage qidx 我已经读完了, 你可以放心覆盖了。"
2103 [[maybe_unused]] auto token = empty[qidx].arrive();
2104 }
2105
2106 // =====
2107 // Epilogue: 将寄存器中的累加结果写回 global memory C
2108 // =====
2109 //
2110 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
2111 //
2112 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
2113 // 这些 half 分布在 d[2][8][4] 数组中:
2114 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
2115 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
2116 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
2117 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
2118 //
2119 // 其中:
2120 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
2121 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
2122 //
2123 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
2124 // +-----+-----+
2125 // | reg[0] | reg[2] | <- rows [row, row+8)
2126 // |(col,+2)|(col+8)|
2127 // +-----+-----+
2128 // | reg[1] | reg[3] | <- rows [row+8, row+16)
2129 // |(col,+2)|(col+8)|
2130 // +-----+-----+
2131 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
2132 //
2133 // 三维遍历:
2134 // m_it=0: rows 0~63, m_it=1: rows 64~127
2135 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
2136 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
2137 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
2138
2139 const int lane = tid % 32;
2140 const int warp = tid / 32;
2141 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。

```

```

2142 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
2143 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
2144 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
2145 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
2146 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
2147 const int row = warp * 16 + lane / 4;
2148 // block_C: 当前 C tile 在 global memory 中的起始地址
2149 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
2150 half *block_C = C + by * BM * N + bx * BN;
2151
2152 #pragma unroll
2153 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2154     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
2155     #pragma unroll
2156     for (int g = 0; g < WGMMMA_N / 16; ++g) {
2157         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
2158         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
2159         // 左上象限: (row, col)
2160         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
2161             d[m_it][g][0];
2162         // 左下象限: (row+8, col)
2163         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
2164             d[m_it][g][1];
2165         // 右上象限: (row, col+8)
2166         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
2167             d[m_it][g][2];
2168         // 右下象限: (row+8, col+8)
2169         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
2170             d[m_it][g][3];
2171     }
2172 }
2173 }
2174 }
2175
2176 #if (defined(NOTES_V2_HAS_WGMMMA))
2177 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
2178 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
2179 // - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
2180 // 以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
2181 // - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
2182 // 对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
2183 // - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
2184 // - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
2185 // - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
2186 //
2187 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
2188
2189 template <int BlockMajorSize, int BlockMinorSize>
2190 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
2191         half *gmem_ptr,
2192         int blocks_height,
2193         int blocks_width) {
2194     void *gmem_address = (void *)gmem_ptr;
2195     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
2196                                     (uint64_t)BlockMajorSize * blocks_height,
2197                                     1, 1, 1};
2198     uint64_t gmem_prob_stride[5] = {
2199         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
2200     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
2201                                    uint32_t(BlockMajorSize), 1, 1, 1};
2202     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
2203     CUresult result = cuTensorMapEncodeTiled(
2204         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,

```

```

2205     gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
2206     CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
2207     CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
2208     if (result != CUDA_SUCCESS)
2209         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
2210 }
2211
2212 __host__ static inline CUTensorMap *allocate_and_create_tensor_map(
2213     half *src, int blocks_height, int blocks_width) {
2214     CUTensorMap *tma_map_d;
2215     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
2216     CUTensorMap tma_map_host;
2217     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
2218     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
2219         cudaMemcpyHostToDevice);
2220     return tma_map_d;
2221 }
2222 #endif /* NOTES_V2_HAS_WGMMA */
2223
2224 // =====
2225 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
2226 // =====
2227 // 面试题点 (FlashAttention 算法) :
2228 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
2229 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
2230 // 2. FA 三板斧:
2231 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqLen 分块 [Bc,d]
2232 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
2233 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
2234 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
2235 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
2236 //    - 优点: 减少 warp 间通信和 shuffle
2237 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
2238 //    for each K,V tile:
2239 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
2240 //        m_new = max(m_old, row_max(S_cur * scale))
2241 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
2242 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
2243 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
2244 //        O_final = O_new / l_final
2245 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
2246 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
2247 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
2248 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
2249 //    - 这是此实现的寄存器布局复用技巧, 不要背成 “所有 MMA A/C fragment
2250 //      都天然同构” 的通用结论
2251 //
2252 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
2253 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
2254 // Grid: ((QKV_seqLen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
2255 // Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
2256 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
2257
2258 // ---- 寄存器填充辅助函数 ----
2259 template <typename T, int M, const int N, const int K = 2>
2260 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
2261     #pragma unroll
2262     for (int i = 0; i < M; ++i)
2263         #pragma unroll
2264         for (int j = 0; j < N; ++j)
2265             #pragma unroll
2266             for (int k = 0; k < K; ++k)
2267                 R[i][j][k] = val;

```



```

2268 }
2269
2270 template <typename T, int M, const int N = 2>
2271 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
2272     #pragma unroll
2273     for (int i = 0; i < M; ++i)
2274     #pragma unroll
2275         for (int j = 0; j < N; ++j)
2276             R[i][j] = val;
2277 }
2278
2279 // =====
2280 // FlashAttention-2 Split-Q Kernel (完整实现)
2281 // =====
2282 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
2283 //
2284 // Tile 设计 (以 kHeadDim=64 为例) :
2285 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
2286 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
2287 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
2288 //
2289 // 执行流程:
2290 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
2291 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
2292 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
2293 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
2294 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
2295 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
2296 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
2297 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
2298 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
2299 //   8) 最终 rescale: O_final = (1/l_final) * O_final
2300 //   9) Epilogue: warp shuffle + 128-bit collective store
2301
2302 template <
2303     const int kHeadDim,           // head dim: 32, 64, 128
2304     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
2305     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
2306     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
2307     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
2308     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
2309     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
2310     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
2311     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
2312     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
2313     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
2314     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
2315     const int kStage,             // pipeline stages for K: 1 or 2
2316     const int kPad>              // padding for bank conflict avoidance
2317 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
2318     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
2319                                     int QKV_seqlen, int QKV_head) {
2320     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
2321     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
2322     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
2323     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
2324     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
2325     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
2326     const float scale = 1.0f / sqrtf((float)kHeadDim);
2327
2328     // Block indexing
2329     const int QKV_batch_id = blockIdx.y / QKV_head;
2330     const int QKV_head_id = blockIdx.y % QKV_head;

```



```

2331 const int Q_tile_id = blockIdx.x;
2332 const int tid = threadIdx.x;
2333 const int warp_id = tid / WARP_SIZE;
2334 const int lane_id = tid % WARP_SIZE;
2335 const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
2336 const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
2337
2338 // Global memory base offsets for this (batch, head)
2339 // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
2340 const int Q_gmem_offset =
2341     (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
2342     kHeadDim;
2343 const int K_gmem_offset = Q_gmem_offset;
2344 const int V_gmem_offset = Q_gmem_offset;
2345 const int O_gmem_offset = Q_gmem_offset;
2346
2347 // Thread-to-smem mapping for cooperative load
2348 int load_smem_Q_Br = tid / (kNumThreads / Br);
2349 int load_smem_Q_d =
2350     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
2351 int load_smem_K_Bc = tid / (kNumThreads / Bc);
2352 int load_smem_K_d =
2353     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2354 int load_smem_V_Bc = tid / (kNumThreads / Bc);
2355 int load_smem_V_d =
2356     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2357
2358 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
2359 if (load_gmem_Q_Br >= QKV_seqlen)
2360     return;
2361
2362 // ---- Shared memory layout ----
2363 extern __shared__ half smem[];
2364 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
2365 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
2366 half *Q_tile_smem = smem;
2367 half *K_tile_smem = Q_tile_smem + Q_tile_size;
2368 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
2369 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
2370 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
2371
2372 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
2373 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
2374 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
2375
2376 // ---- Online Softmax persistent state ----
2377 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
2378 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
2379 float lane_block_row_max_old[kValTileSeqLenQ][2];
2380 float lane_block_row_sum_old[kValTileSeqLenQ][2];
2381 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
2382 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
2383
2384 // ---- Register allocation ----
2385 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
2386 uint32_t R_K[kValTileSeqLenK][2]; // K regs
2387 uint32_t R_V[kValTileHeadDimV][2]; // V regs
2388 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
2389 // 对单个 m16n8k16 tile 而言:
2390 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
2391 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
2392 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
2393 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)

```

```

2394 uint32_t R_0[kValTileSeqLenP][kValTileHeadDimV][2]; // 0 for current tile
2395 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
2396         [2]; // 0 accumulator (final output)
2397
2398 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2399 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
2400
2401 // =====
2402 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
2403 // =====
2404 {
2405     int load_gmem_Q_addr =
2406         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
2407     uint32_t load_smem_Q_ptr =
2408         smem_Q_base_ptr +
2409         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
2410 #pragma unroll
2411     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
2412         CP_ASYNC.CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
2413     }
2414     CP_ASYNC.COMMIT_GROUP();
2415 }
2416
2417 // =====
2418 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
2419 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
2420 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
2421 // =====
2422 if constexpr (kStage > 1) {
2423 #pragma unroll
2424     for (int stage = 0; stage < (kStage - 1); ++stage) {
2425         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
2426         int load_gmem_K_addr =
2427             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2428         uint32_t load_smem_K_ptr =
2429             smem_K_base_ptr +
2430             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2431              load_smem_K_d) *
2432             sizeof(half);
2433 #pragma unroll
2434         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2435             CP_ASYNC.CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2436         }
2437         CP_ASYNC.COMMIT_GROUP();
2438     }
2439     CP_ASYNC.WAIT_GROUP(kStage - 2);
2440     __syncthreads();
2441 }
2442
2443 // =====
2444 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
2445 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
2446 // =====
2447 #pragma unroll 1
2448 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
2449     int smem_sel = tile_K_seqlen % kStage;
2450     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
2451
2452     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
2453     if constexpr (kStage > 1) {
2454         // 只有 kStage>1 才能真正做 K 的 pipeline:
2455         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
2456         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。

```

```

2457 // Load current V tile (no pipeline for V — one stage is enough)
2458 {
2459     int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
2460     int load_gmem_V_addr =
2461         V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2462     uint32_t load_smem_V_ptr =
2463         smem_V_base_ptr +
2464         (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
2465 #pragma unroll
2466     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2467         CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
2468     }
2469     CP_ASYNC_COMMIT_GROUP();
2470 }
2471
2472 // Prefetch next K tile (pipelined)
2473 if ((tile_K_seqlen + 1) < Tc) {
2474     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
2475     int load_gmem_K_addr =
2476         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2477     uint32_t load_smem_K_ptr =
2478         smem_K_base_ptr +
2479         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2480          load_smem_K_d) *
2481         sizeof(half);
2482 #pragma unroll
2483     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2484         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2485     }
2486     CP_ASYNC_COMMIT_GROUP();
2487 }
2488 }
2489
2490 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
2491 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2492 #pragma unroll
2493 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
2494     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2495     #pragma unroll
2496     for (int i = 0; i < kValTileSeqLenQ; ++i) {
2497         int warp_smem_Q_Br =
2498             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
2499         int lane_smem_Q_Br =
2500             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2501         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
2502         uint32_t lane_smem_Q_ptr =
2503             smem_Q_base_ptr +
2504             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
2505         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2506                    lane_smem_Q_ptr);
2507     }
2508
2509     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2510     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2511     #pragma unroll
2512     for (int j = 0; j < kValTileSeqLenK; ++j) {
2513         int warp_smem_K_Bc =
2514             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
2515         int lane_smem_K_Bc =
2516             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2517         int lane_smem_K_d =
2518             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2519         uint32_t lane_smem_K_ptr =

```

```

2520         smem_K_base_ptr +
2521         (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
2522         lane_smem_K_d) *
2523         sizeof(half);
2524     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2525 }
2526
2527 // MMA: S[tile] += Q[tile] @ K^T[tile]
2528 #pragma unroll
2529 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2530 #pragma unroll
2531     for (int j = 0; j < kValTileSeqLenK; ++j) {
2532         MMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
2533                 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2534                 R_S[i][j][1]);
2535     }
2536 }
2537 } // end loop over d
2538 __syncthreads();
2539
2540 // =====
2541 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
2542 // =====
2543 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2544 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
2545 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
2546 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
2547 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
2548 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
2549 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2550 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
2551 // kValTileSeqLenK tiles.
2552
2553 float lane_row_max_new[kValTileSeqLenQ][2];
2554 float lane_row_sum_new[kValTileSeqLenQ][2];
2555 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
2556 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2557
2558 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
2559 #pragma unroll
2560 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2561 #pragma unroll
2562     for (int j = 0; j < kValTileSeqLenK; ++j) {
2563         // Extract half values from R_S registers (C matrix fragment layout)
2564         float2 t_reg_S_0 =
2565             __half2float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
2566         float2 t_reg_S_1 =
2567             __half2float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
2568         // S = (Q@K^T) * scale
2569         float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2570         float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2571         lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2572         lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2573     }
2574     // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2575     // {0,4,8,...,28} hold valid data)
2576     lane_row_max_new[i][0] =
2577         warp_reduce_max<4, float>(lane_row_max_new[i][0]);
2578     lane_row_max_new[i][1] =
2579         warp_reduce_max<4, float>(lane_row_max_new[i][1]);
2580 }
2581
2582 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S

```

```

2583 // 面试关键点: 这里将 P 写回 R_S 寄存器!
2584 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2585 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
2586 #pragma unroll
2587 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2588     // m_new = max(m_old, m_cur)
2589     float block_row_max_new_0 =
2590         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2591     float block_row_max_new_1 =
2592         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2593
2594 #pragma unroll
2595 for (int j = 0; j < kValTileSeqLenK; ++j) {
2596     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2597     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2598
2599     // P = exp(S * scale - m_new), 用 fma 保证精度
2600     t_reg_S_0.x =
2601         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2602     t_reg_S_0.y =
2603         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2604     t_reg_S_1.x =
2605         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2606     t_reg_S_1.y =
2607         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2608
2609     // Accumulate row sums
2610     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2611     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2612
2613     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2614     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2615     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
2616 }
2617
2618 // Warp-level reduce sum (warp_size = 4, same as max)
2619 lane_row_sum_new[i][0] =
2620     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
2621 lane_row_sum_new[i][1] =
2622     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
2623 }
2624 __syncthreads();
2625
2626 // =====
2627 // 3d: P@V - P[Br,Bc] @ V[Bc,d] = 0[Br,d]
2628 // =====
2629 // Wait for V to be ready before computing P@V
2630 if constexpr (kStage > 1) {
2631     if ((tile_K_seqLen + 1) < Tc) {
2632         CP_ASYNC_WAIT_GROUP(1);
2633     } else {
2634         CP_ASYNC_WAIT_GROUP(0);
2635     }
2636 } else {
2637     CP_ASYNC_WAIT_GROUP(0);
2638 }
2639 __syncthreads();
2640
2641 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
2642
2643 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2644 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
2645 #pragma unroll

```

```

2646 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
2647     // ldmatrix.x2.trans: load V[Bc,d] with transposition
2648     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
2649     // transposed ldmatrix
2650 #pragma unroll
2651     for (int j = 0; j < kValTileHeadDimV; ++j) {
2652         int warp_smem_V_d =
2653             warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2654         int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2655         int lane_smem_V_d = warp_smem_V_d;
2656         uint32_t lane_smem_V_ptr =
2657             smem_V_base_ptr +
2658             (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
2659         LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2660     }
2661
2662     // P matrix layout in R_S[i][j][2]:
2663     // MMA = m16n8kl6, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2664     // | 64x64 | warp_KV 0 |
2665     // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2666     // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2667     // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2668     // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2669     // tile_V_Bc selects which 16-column slice of P to use:
2670     // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
2671     // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
2672     // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
2673     // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
2674     // 对应的 MMA A fragment 布局可以这样记:
2675     // - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
2676     // - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
2677     // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
2678     // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
2679     // fragment 都无条件成立的通用结论。layout转换逻辑:
2680     // C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
2681     int w = tile_V_Bc * 2;
2682 #pragma unroll
2683     for (int i = 0; i < kValTileSeqLenP; ++i) {
2684 #pragma unroll
2685         for (int j = 0; j < kValTileHeadDimV; ++j) {
2686             HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
2687                 R_S[i][w][0], R_S[i][w][1], R_S[i][w+1][0], R_S[i][w+1][1], // A fragment = P
2688                 R_V[j][0], R_V[j][1], // B fragment = V
2689                 R_0[i][j][0], R_0[i][j][1]); // C fragment output
2690         }
2691     }
2692 } // end for tile_V_Bc
2693 __syncthreads();
2694
2695 // =====
2696 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
2697 // =====
2698 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
2699 #pragma unroll
2700 for (int i = 0; i < kValTileSeqLenP; ++i) {
2701     float block_row_max_new_0 = lane_row_max_new[i][0];
2702     float block_row_max_new_1 = lane_row_max_new[i][1];
2703     float block_row_sum_new_0 = lane_row_sum_new[i][0];
2704     float block_row_sum_new_1 = lane_row_sum_new[i][1];
2705
2706     float block_row_max_old_0 = lane_block_row_max_old[i][0];
2707     float block_row_max_old_1 = lane_block_row_max_old[i][1];
2708

```

```

2709     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
2710     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
2711
2712     // Handle first iteration: m_old = -inf, need to use m_new directly
2713     block_row_max_old_0 =
2714         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
2715     block_row_max_old_1 =
2716         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
2717
2718     float rescale_o_factor_0 =
2719         __expf(block_row_max_old_0 - block_row_max_new_0);
2720     float rescale_o_factor_1 =
2721         __expf(block_row_max_old_1 - block_row_max_new_1);
2722
2723     // Rescale O_old + Add P@V in one fused step
2724     #pragma unroll
2725     for (int j = 0; j < kValTileHeadDimV; ++j) {
2726         // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
2727         //   reg[0] -> rows 0~7 的 {c0,c1}
2728         //   reg[1] -> rows 8~15 的 {c2,c3}
2729         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
2730         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
2731         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2732         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2733
2734         // O_new = exp(m_old - m_new) * O_old + P@V (fused multiply-add)
2735         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
2736         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
2737         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
2738         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
2739
2740         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2741         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2742     }
2743
2744     // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
2745     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
2746     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
2747     lane_block_row_sum_old[i][0] = __fmaf_rn(
2748         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
2749     lane_block_row_sum_old[i][1] = __fmaf_rn(
2750         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
2751
2752     // Update m
2753     lane_block_row_max_old[i][0] = block_row_max_new_0;
2754     lane_block_row_max_old[i][1] = block_row_max_new_1;
2755 }
2756
2757 // Wait for next K tile to be ready in smem before next iteration
2758 if constexpr (kStage > 1) {
2759     if ((tile_K_seqlen + 1) < Tc) {
2760         CP_ASYNC_WAIT_GROUP(0);
2761     }
2762     __syncthreads();
2763 }
2764 } // end outer loop over K seqlen
2765 __syncthreads();
2766
2767 // =====
2768 // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
2769 // =====
2770 #pragma unroll
2771 for (int i = 0; i < kValTileSeqLenP; ++i) {

```



```

2772     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
2773     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2774 #pragma unroll
2775     for (int j = 0; j < kValTileHeadDimV; ++j) {
2776         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2777         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2778         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2779         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2780         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2781         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2782         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2783         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2784     }
2785 }
2786
2787 // =====
2788 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
2789 // =====
2790 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
2791 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2792 #pragma unroll
2793     for (int i = 0; i < kValTileSeqLenP; ++i) {
2794 #pragma unroll
2795         for (int j = 0; j < kValTileHeadDimV; ++j) {
2796             uint32_t R_Z[2][4];
2797             R_Z[0][0] = R_D[i][j][0];
2798             R_Z[1][0] = R_D[i][j][1];
2799             R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2800             R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2801             R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2802             R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2803             R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2804             R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
2805
2806             // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2807             if (lane_id % 4 == 0) {
2808                 int store_warp_regs_0_Br =
2809                     warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
2810                 int store_lane_gmem_0_Br =
2811                     Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2812                 int store_warp_regs_0_d =
2813                     warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2814                 int store_lane_gmem_0_d = store_warp_regs_0_d;
2815                 int store_gmem_0_addr_0 = 0_gmem_offset +
2816                     (store_lane_gmem_0_Br + 0) * kHeadDim +
2817                     store_lane_gmem_0_d;
2818                 int store_gmem_0_addr_1 = 0_gmem_offset +
2819                     (store_lane_gmem_0_Br + 8) * kHeadDim +
2820                     store_lane_gmem_0_d;
2821                 // LDST128BITS = reinterpret_cast<float4*>
2822                 *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2823                     *reinterpret_cast<float4*>(&R_Z[0][0]);
2824                 *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2825                     *reinterpret_cast<float4*>(&R_Z[1][0]);
2826             }
2827         }
2828     }
2829 }
2830
2831 // =====
2832 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
2833 // =====
2834

```

```

2835 static inline void check(cudaError_t err, const char *msg) {
2836     if (err != cudaSuccess) {
2837         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
2838         exit(EXIT_FAILURE);
2839     }
2840 }
2841
2842
2843 static void test_block_reduce(int N) {
2844
2845     srand(42);
2846     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2847     for (int i = 0; i < N; i++)
2848         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2849
2850     // CPU reference: sum of all elements
2851     double ref = 0.0;
2852     for (int i = 0; i < N; i++) ref += (double)h_a[i];
2853
2854     float *d_a, *d_y;
2855     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
2856     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
2857
2858     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
2859     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
2860
2861     dim3 block(128);
2862     dim3 grid((N + 127) / 128);
2863     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
2864     check(cudaGetLastError(), "blockreduce launch");
2865     check(cudaDeviceSynchronize(), "blockreduce sync");
2866
2867     float result;
2868     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
2869
2870     float err = fabsf(result - (float)ref);
2871     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
2872         err < 1e-2f ? "PASS" : "FAIL");
2873
2874     free(h_a);
2875     cudaFree(d_a); cudaFree(d_y);
2876 }
2877
2878
2879 static void test_dot(int N) {
2880
2881     srand(42);
2882     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2883     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2884     for (int i = 0; i < N; i++) {
2885         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2886         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2887     }
2888
2889     // CPU reference
2890     double ref = 0.0;
2891     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
2892
2893     float *d_a, *d_b, *d_y;
2894     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
2895     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
2896     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
2897

```

```

2898 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
2899 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
2900 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
2901
2902 dim3 block(128);
2903 dim3 grid((N + 127) / 128);
2904 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
2905 check(cudaGetLastError(), "dot launch");
2906 check(cudaDeviceSynchronize(), "dot sync");
2907
2908 float result;
2909 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
2910
2911 float err = fabsf(result - (float)ref);
2912 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
2913        err < 1e-2f ? "PASS" : "FAIL");
2914
2915 // ---- Dot Vec4 ----
2916 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
2917 dim3 block_v4(32);
2918 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
2919 check(cudaGetLastError(), "dot_vec4 launch");
2920 check(cudaDeviceSynchronize(), "dot_vec4 sync");
2921
2922 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
2923 float err_v4 = fabsf(result - (float)ref);
2924 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
2925
2926 free(h_a); free(h_b);
2927 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
2928 }
2929
2930
2931 static void test_relu(int N) {
2932     srand(42);
2933     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2934     float *h_y = (float *)malloc((size_t)N * sizeof(float));
2935     for (int i = 0; i < N; i++)
2936         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2937
2938     float *d_x, *d_y;
2939     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
2940     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
2941     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
2942
2943     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
2944     dim3 block256(256);
2945     dim3 grid256((N + 255) / 256);
2946     relu<<<grid256, block256>>>(d_x, d_y, N);
2947     check(cudaGetLastError(), "relu launch");
2948     check(cudaDeviceSynchronize(), "relu sync");
2949     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
2950     float max_err = 0.0f;
2951     for (int i = 0; i < N; i++) {
2952         float expected = fmaxf(0.0f, h_x[i]);
2953         float err = fabsf(h_y[i] - expected);
2954         if (err > max_err) max_err = err;
2955     }
2956     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2957
2958     dim3 block64(64);
2959     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
2960     check(cudaGetLastError(), "relu_vec4 launch");

```

```

2961 check(cudaDeviceSynchronize(), "relu_vec4 sync");
2962 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
2963 max_err = 0.0f;
2964 for (int i = 0; i < N; i++) {
2965     float expected = fmaxf(0.0f, h_x[i]);
2966     float err = fabsf(h_y[i] - expected);
2967     if (err > max_err) max_err = err;
2968 }
2969 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2970
2971 free(h_x); free(h_y);
2972 cudaFree(d_x); cudaFree(d_y);
2973 }
2974
2975
2976 static void test_elementwise(int N) {
2977     srand(42);
2978     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2979     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2980     for (int i = 0; i < N; i++) {
2981         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2982         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2983     }
2984
2985     float *d_a, *d_b, *d_c;
2986     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
2987     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
2988     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
2989     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
2990     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
2991
2992     dim3 block256(256);
2993     dim3 grid256((N + 255) / 256);
2994     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
2995     check(cudaGetLastError(), "eadd launch");
2996     check(cudaDeviceSynchronize(), "eadd sync");
2997     float *h_c = (float *)malloc((size_t)N * sizeof(float));
2998     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
2999     float max_err = 0.0f;
3000     for (int i = 0; i < N; i++) {
3001         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3002         if (err > max_err) max_err = err;
3003     }
3004     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3005
3006     dim3 block64(64);
3007     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
3008     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
3009     check(cudaGetLastError(), "eadd_vec4 launch");
3010     check(cudaDeviceSynchronize(), "eadd_vec4 sync");
3011     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
3012     max_err = 0.0f;
3013     for (int i = 0; i < N; i++) {
3014         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
3015         if (err > max_err) max_err = err;
3016     }
3017     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3018
3019     free(h_a); free(h_b); free(h_c);
3020     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3021 }
3022
3023

```

```

3024 static void test_histogram(int N) {
3025     srand(42);
3026     int BINS = 16;
3027     int *h_hist = (int *)calloc(BINS, sizeof(int));
3028     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
3029     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
3030     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
3031     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
3032
3033     int *d_idx, *d_hist;
3034     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
3035     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
3036     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
3037     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
3038
3039     dim3 block256(256);
3040     dim3 grid256((N + 255) / 256);
3041     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
3042     check(cudaGetLastError(), "histogram launch");
3043     check(cudaDeviceSynchronize(), "histogram sync");
3044     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
3045
3046     float max_err = 0.0f;
3047     for (int i = 0; i < BINS; i++) {
3048         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
3049         if (err > max_err) max_err = err;
3050     }
3051     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3052
3053     free(h_hist); free(h_hist_ref); free(h_idx);
3054     cudaFree(d_idx); cudaFree(d_hist);
3055 }
3056
3057 static void test_merge_attn_states(int num_tokens, int num_heads,
3058                                   int head_size) {
3059
3060     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
3061     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
3062
3063     float *h_prefix_out = (float *)malloc(out_size);
3064     float *h_suffix_out = (float *)malloc(out_size);
3065     float *h_prefix_lse = (float *)malloc(lse_size);
3066     float *h_suffix_lse = (float *)malloc(lse_size);
3067     float *h_output_ref = (float *)malloc(out_size);
3068
3069     srand(42);
3070     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3071         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3072         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3073     }
3074     for (int i = 0; i < num_heads * num_tokens; i++) {
3075         h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3076         h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
3077     }
3078
3079     // CPU reference: 与 kernel 完全相同的浮点计算
3080     for (int t = 0; t < num_tokens; t++) {
3081         for (int h = 0; h < num_heads; h++) {
3082             float p_lse = h_prefix_lse[h * num_tokens + t];
3083             float s_lse = h_suffix_lse[h * num_tokens + t];
3084             p_lse = isinf(p_lse) ? -INFINITY : p_lse;
3085             s_lse = isinf(s_lse) ? -INFINITY : s_lse;

```

```

3087
3088     float max_lse = fmaxf(p_lse, s_lse);
3089     p_lse -= max_lse;
3090     s_lse -= max_lse;
3091     float p_se = expf(p_lse);
3092     float s_se = expf(s_lse);
3093     float p_scale = p_se / (p_se + s_se);
3094     float s_scale = s_se / (p_se + s_se);
3095
3096     int head_off = t * num_heads * head_size + h * head_size;
3097     for (int d = 0; d < head_size; d++) {
3098         h_output_ref[head_off + d] =
3099             h_prefix_out[head_off + d] * p_scale +
3100             h_suffix_out[head_off + d] * s_scale;
3101     }
3102 }
3103 }
3104
3105 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
3106 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
3107 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
3108 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
3109 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
3110 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
3111
3112 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
3113                 cudaMemcpyHostToDevice),
3114        "merge_attn H2D p_out");
3115 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
3116                 cudaMemcpyHostToDevice),
3117        "merge_attn H2D s_out");
3118 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3119                 cudaMemcpyHostToDevice),
3120        "merge_attn H2D p_lse");
3121 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
3122                 cudaMemcpyHostToDevice),
3123        "merge_attn H2D s_lse");
3124
3125 int threads_per_head = head_size / 4;
3126 int total_threads = num_tokens * num_heads * threads_per_head;
3127 dim3 block(128);
3128 dim3 grid((total_threads + 127) / 128);
3129 merge_attn_states<<<grid, block>>>(&
3130     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3131     num_tokens, num_heads, head_size);
3132 check(cudaGetLastError(), "merge_attn launch");
3133 check(cudaDeviceSynchronize(), "merge_attn sync");
3134
3135 float *h_output = (float *)malloc(out_size);
3136 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3137        "merge_attn D2H");
3138
3139 float max_err = 0.0f;
3140 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
3141     float err = fabsf(h_output[i] - h_output_ref[i]);
3142     if (err > max_err) max_err = err;
3143 }
3144 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
3145        max_err < 1e-4f ? "PASS" : "FAIL");
3146
3147 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
3148 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
3149 h_suffix_lse[0] = 0.0f;

```

```

3150 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
3151               cudaMemcpyHostToDevice),
3152       "merge_attn H2D inf lse");
3153 merge_attn_states<<<grid, block>>>{
3154     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
3155     num_tokens, num_heads, head_size);
3156 check(cudaGetLastError(), "merge_attn inf launch");
3157 check(cudaDeviceSynchronize(), "merge_attn inf sync");
3158 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
3159       "merge_attn inf D2H");
3160
3161 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
3162 float inf_err = 0.0f;
3163 for (int d = 0; d < head_size; d++) {
3164     float err = fabsf(h_output[d] - h_suffix_out[d]);
3165     if (err > inf_err) inf_err = err;
3166 }
3167 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,
3168       inf_err < 1e-4f ? "PASS" : "FAIL");
3169
3170 free(h_prefix_out);
3171 free(h_suffix_out);
3172 free(h_prefix_lse);
3173 free(h_suffix_lse);
3174 free(h_output_ref);
3175 free(h_output);
3176 cudaFree(d_prefix_out);
3177 cudaFree(d_suffix_out);
3178 cudaFree(d_prefix_lse);
3179 cudaFree(d_suffix_lse);
3180 cudaFree(d_output);
3181 }
3182
3183
3184 static void test_softmax(int N) {
3185     // Use N = blockDim.x so one block processes all elements as one token.
3186     constexpr int NUM_THREADS = 256;
3187     if (N != NUM_THREADS) {
3188         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
3189         return;
3190     }
3191
3192     srand(42);
3193     float *h_x = (float *)malloc((size_t)N * sizeof(float));
3194     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
3195     for (int i = 0; i < N; i++)
3196         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
3197
3198     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
3199     float max_val = -FLT_MAX;
3200     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
3201     double sum_exp = 0.0;
3202     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
3203     for (int i = 0; i < N; i++)
3204         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
3205
3206     float *d_x, *d_y;
3207     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
3208     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
3209
3210     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
3211
3212     dim3 block(256);

```



```

3213 dim3 grid(1); // one token covering all N elements
3214 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3215 check(cudaGetLastError(), "softmax launch");
3216 check(cudaDeviceSynchronize(), "softmax sync");
3217
3218 float *h_y = (float *)malloc((size_t)N * sizeof(float));
3219 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
3220
3221 float max_err = 0.0f;
3222 for (int i = 0; i < N; i++) {
3223     float err = fabsf(h_y[i] - h_y_ref[i]);
3224     if (err > max_err) max_err = err;
3225 }
3226 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3227
3228 // ---- Safe Softmax ----
3229 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3230 check(cudaGetLastError(), "safe_softmax launch");
3231 check(cudaDeviceSynchronize(), "safe_softmax sync");
3232 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
3233 max_err = 0.0f;
3234 for (int i = 0; i < N; i++) {
3235     float err = fabsf(h_y[i] - h_y_ref[i]);
3236     if (err > max_err) max_err = err;
3237 }
3238 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3239
3240 // ---- Naive Softmax ----
3241 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3242 check(cudaGetLastError(), "naive_softmax launch");
3243 check(cudaDeviceSynchronize(), "naive_softmax sync");
3244 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
3245 max_err = 0.0f;
3246 for (int i = 0; i < N; i++) {
3247     float err = fabsf(h_y[i] - h_y_ref[i]);
3248     if (err > max_err) max_err = err;
3249 }
3250 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3251
3252 free(h_x); free(h_y); free(h_y_ref);
3253 cudaFree(d_x); cudaFree(d_y);
3254 }
3255
3256
3257 static void test_rms_norm(int N, int K) {
3258
3259     srand(42);
3260     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
3261     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
3262     for (int i = 0; i < N * K; i++)
3263         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3264     float g = 1.5f; // gain
3265
3266     // CPU reference: y = (x / rms(x)) * g
3267     float epsilon = 1e-5f;
3268     for (int n = 0; n < N; n++) {
3269         double sum_sq = 0.0;
3270         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
3271         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
3272         for (int k = 0; k < K; k++)
3273             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
3274     }
3275

```

```

3276 float *d_x, *d_y;
3277 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
3278 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
3279 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
3280
3281 dim3 block(128);
3282 dim3 grid(N);
3283 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
3284 check(cudaGetLastError(), "rmsnorm launch");
3285 check(cudaDeviceSynchronize(), "rmsnorm sync");
3286
3287 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
3288 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
3289
3290 float max_err = 0.0f;
3291 for (int i = 0; i < N * K; i++) {
3292     float err = fabsf(h_y[i] - h_y_ref[i]);
3293     if (err > max_err) max_err = err;
3294 }
3295 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3296
3297 // ---- RMS Norm Vec4 ----
3298 dim3 block_rv4(32);
3299 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
3300 check(cudaGetLastError(), "rmsnorm_vec4 launch");
3301 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
3302 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
3303 max_err = 0.0f;
3304 for (int i = 0; i < N * K; i++) {
3305     float err = fabsf(h_y[i] - h_y_ref[i]);
3306     if (err > max_err) max_err = err;
3307 }
3308 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3309
3310 free(h_x); free(h_y); free(h_y_ref);
3311 cudaFree(d_x); cudaFree(d_y);
3312 }
3313
3314
3315 static void test_layer_norm(int N, int K) {
3316
3317     srand(42);
3318     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
3319     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
3320     for (int i = 0; i < N * K; i++)
3321         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3322     float g = 1.5f, b = 0.3f; // gain and bias
3323
3324     // CPU reference: y = ((x - mean) / std) * g + b
3325     float epsilon = 1e-5f;
3326     for (int n = 0; n < N; n++) {
3327         double sum = 0.0;
3328         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
3329         float mean = (float)sum / (float)K;
3330         double sum_sq = 0.0;
3331         for (int k = 0; k < K; k++) {
3332             float diff = h_x[n * K + k] - mean;
3333             sum_sq += (double)diff * (double)diff;
3334         }
3335         float std = sqrtf((float)sum_sq / (float)K + epsilon);
3336         for (int k = 0; k < K; k++)
3337             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
3338     }

```

```

3339
3340 float *d_x, *d_y;
3341 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
3342 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
3343 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
3344
3345 dim3 block(128);
3346 dim3 grid(N);
3347 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
3348 check(cudaGetLastError(), "layernorm launch");
3349 check(cudaDeviceSynchronize(), "layernorm sync");
3350
3351 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
3352 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
3353
3354 float max_err = 0.0f;
3355 for (int i = 0; i < N * K; i++) {
3356     float err = fabsf(h_y[i] - h_y_ref[i]);
3357     if (err > max_err) max_err = err;
3358 }
3359 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3360
3361 // ---- Layer Norm Vec4 ----
3362 dim3 block_lv4(32);
3363 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
3364 check(cudaGetLastError(), "layernorm_vec4 launch");
3365 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
3366 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
3367 max_err = 0.0f;
3368 for (int i = 0; i < N * K; i++) {
3369     float err = fabsf(h_y[i] - h_y_ref[i]);
3370     if (err > max_err) max_err = err;
3371 }
3372 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3373
3374 free(h_x); free(h_y); free(h_y_ref);
3375 cudaFree(d_x); cudaFree(d_y);
3376 }
3377
3378
3379 static void test_rope(int seq_len, int N) {
3380
3381     int total_pairs = seq_len * N;
3382     int total_elems = total_pairs * 2;
3383     size_t size = (size_t)total_elems * sizeof(float);
3384
3385     float *h_x = (float *)malloc(size);
3386     float *h_y_ref = (float *)malloc(size);
3387
3388     srand(42);
3389     for (int i = 0; i < total_elems; i++)
3390         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3391
3392     // CPU reference: 2D rotation for each pair
3393     for (int idx = 0; idx < total_pairs; idx++) {
3394         int token_pos = idx / N;
3395         int token_idx = idx % N;
3396         float x1 = h_x[idx * 2];
3397         float x2 = h_x[idx * 2 + 1];
3398         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
3399         float angle = (float)token_pos * theta;
3400         float cos_v = cosf(angle);
3401         float sin_v = sinf(angle);

```

```

3402     h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
3403     h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
3404 }
3405
3406 float *d_x, *d_y;
3407 check(cudaMalloc(&d_x, size), "rope alloc X");
3408 check(cudaMalloc(&d_y, size), "rope alloc Y");
3409 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
3410
3411 dim3 block(256);
3412 dim3 grid((total_pairs + 255) / 256);
3413 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
3414 check(cudaGetLastError(), "rope launch");
3415 check(cudaDeviceSynchronize(), "rope sync");
3416
3417 float *h_y = (float *)malloc(size);
3418 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
3419
3420 float max_err = 0.0f;
3421 for (int i = 0; i < total_elems; i++) {
3422     float err = fabsf(h_y[i] - h_y_ref[i]);
3423     if (err > max_err) max_err = err;
3424 }
3425 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3426
3427 free(h_x);
3428 free(h_y);
3429 free(h_y_ref);
3430 cudaFree(d_x);
3431 cudaFree(d_y);
3432 }
3433
3434
3435 static void test_mat_transpose(int row, int col) {
3436
3437     size_t size_in = (size_t)row * col * sizeof(float);
3438     size_t size_out = (size_t)col * row * sizeof(float);
3439
3440     float *h_x = (float *)malloc(size_in);
3441     float *h_y_ref = (float *)malloc(size_out);
3442
3443     srand(42);
3444     for (int i = 0; i < row * col; i++)
3445         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3446
3447     // CPU reference: y[j][i] = x[i][j] (row-major)
3448     for (int i = 0; i < row; i++)
3449         for (int j = 0; j < col; j++)
3450             h_y_ref[j * row + i] = h_x[i * col + j];
3451
3452     float *d_x, *d_y;
3453     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
3454     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
3455     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
3456
3457     dim3 block(16, 16);
3458     dim3 grid((col + 15) / 16, (row + 15) / 16);
3459     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
3460     check(cudaGetLastError(), "mattrans launch");
3461     check(cudaDeviceSynchronize(), "mattrans sync");
3462
3463     float *h_y = (float *)malloc(size_out);
3464     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");

```

```

3465
3466 float max_err = 0.0f;
3467 for (int i = 0; i < col * row; i++) {
3468     float err = fabsf(h_y[i] - h_y_ref[i]);
3469     if (err > max_err) max_err = err;
3470 }
3471 printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3472
3473 free(h_x);
3474 free(h_y);
3475 free(h_y_ref);
3476 cudaFree(d_x);
3477 cudaFree(d_y);
3478 }
3479
3480
3481 static void test_mat_transpose_padded(int row, int col) {
3482
3483     size_t size_in = (size_t)row * col * sizeof(float);
3484     size_t size_out = (size_t)col * row * sizeof(float);
3485
3486     float *h_x = (float *)malloc(size_in);
3487     float *h_y_ref = (float *)malloc(size_out);
3488
3489     srand(42);
3490     for (int i = 0; i < row * col; i++)
3491         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3492
3493     // CPU reference: y[j][i] = x[i][j] (row-major)
3494     for (int i = 0; i < row; i++)
3495         for (int j = 0; j < col; j++)
3496             h_y_ref[j * row + i] = h_x[i * col + j];
3497
3498     float *d_x, *d_y;
3499     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
3500     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
3501     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
3502
3503     dim3 block(16, 16);
3504     dim3 grid((col + 15) / 16, (row + 63) / 64);
3505     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
3506     check(cudaGetLastError(), "mattrans_padded launch");
3507     check(cudaDeviceSynchronize(), "mattrans_padded sync");
3508
3509     float *h_y = (float *)malloc(size_out);
3510     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
3511
3512     float max_err = 0.0f;
3513     for (int i = 0; i < col * row; i++) {
3514         float err = fabsf(h_y[i] - h_y_ref[i]);
3515         if (err > max_err) max_err = err;
3516     }
3517     printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3518
3519     free(h_x);
3520     free(h_y);
3521     free(h_y_ref);
3522     cudaFree(d_x);
3523     cudaFree(d_y);
3524 }
3525
3526
3527 static void test_sgemv(int M, int K) {

```

```

3528
3529 srand(42);
3530 float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
3531 float *h_x = (float *)malloc((size_t)K * sizeof(float));
3532 float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3533 for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3534 for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3535
3536 // CPU reference: y[m] = sum_k A[m,k] * x[k]
3537 for (int m = 0; m < M; m++) {
3538     double sum = 0.0;
3539     for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
3540     h_y_ref[m] = (float)sum;
3541 }
3542
3543 float *d_a, *d_x, *d_y;
3544 check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
3545 check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
3546 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
3547
3548 check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
3549 check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
3550
3551 dim3 block(32, 4);
3552 dim3 grid((M + 3) / 4);
3553 sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
3554 check(cudaGetLastError(), "sgemv launch");
3555 check(cudaDeviceSynchronize(), "sgemv sync");
3556
3557 float *h_y = (float *)malloc((size_t)M * sizeof(float));
3558 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
3559
3560 float max_err = 0.0f;
3561 for (int m = 0; m < M; m++) {
3562     float err = fabsf(h_y[m] - h_y_ref[m]);
3563     if (err > max_err) max_err = err;
3564 }
3565 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3566
3567 // ---- SGEMV K32 ----
3568 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
3569 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
3570 check(cudaGetLastError(), "sgemv_k32 launch");
3571 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
3572
3573 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
3574
3575 max_err = 0.0f;
3576 for (int m = 0; m < M; m++) {
3577     float err = fabsf(h_y[m] - h_y_ref[m]);
3578     if (err > max_err) max_err = err;
3579 }
3580 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3581
3582 // ---- SGEMV K16 ----
3583 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3584 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3585
3586 int K16 = 16;
3587 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
3588 h_x = (float *)malloc((size_t)K16 * sizeof(float));
3589 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3590 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

```

```

3591 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3592
3593 for (int m = 0; m < M; m++) {
3594     double sum = 0.0;
3595     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
3596     h_y_ref[m] = (float)sum;
3597 }
3598
3599 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
3600 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
3601 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
3602
3603 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
3604 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
3605
3606 dim3 grid_k16((M + 7) / 8);
3607 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
3608 check(cudaGetLastError(), "sgemv_k16 launch");
3609 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
3610
3611 h_y = (float *)malloc((size_t)M * sizeof(float));
3612 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
3613
3614 max_err = 0.0f;
3615 for (int m = 0; m < M; m++) {
3616     float err = fabsf(h_y[m] - h_y_ref[m]);
3617     if (err > max_err) max_err = err;
3618 }
3619 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3620
3621 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3622 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3623 }
3624
3625
3626 static void test_sgemm(int M, int N, int K) {
3627
3628     size_t size_a = (size_t)M * K * sizeof(float);
3629     size_t size_b = (size_t)K * N * sizeof(float);
3630     size_t size_c = (size_t)M * N * sizeof(float);
3631
3632     float *h_a = (float *)malloc(size_a);
3633     float *h_b = (float *)malloc(size_b);
3634     float *h_c_ref = (float *)malloc(size_c);
3635
3636     srand(42);
3637     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3638     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3639
3640     float *d_a, *d_b, *d_c;
3641     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
3642     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
3643     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
3644
3645     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
3646     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
3647
3648     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
3649     cublasHandle_t handle;
3650     cublasCreate(&handle);
3651     float alpha = 1.0f, beta = 0.0f;
3652     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3653                 &alpha, d_b, N, d_a, K, &beta, d_c, N);

```



```

3654 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
3655
3656 // Kernel
3657 cudaEvent_t start, stop;
3658 cudaEventCreate(&start);
3659 cudaEventCreate(&stop);
3660
3661 dim3 block(32, 32);
3662 dim3 grid((N + 31) / 32, (M + 31) / 32);
3663 sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
3664 check(cudaGetLastError(), "sgemm launch");
3665 check(cudaDeviceSynchronize(), "sgemm sync");
3666
3667 float *h_c = (float *)malloc(size_c);
3668 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
3669
3670 // Verify
3671 float max_err = 0.0f;
3672 for (int i = 0; i < M * N; i++) {
3673     float err = fabsf(h_c[i] - h_c_ref[i]);
3674     if (err > max_err) max_err = err;
3675 }
3676 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3677
3678 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
3679
3680 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
3681 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
3682 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
3683 check(cudaGetLastError(), "sgemm_vec4 launch");
3684 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
3685
3686 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
3687
3688 max_err = 0.0f;
3689 for (int i = 0; i < M * N; i++) {
3690     float err = fabsf(h_c[i] - h_c_ref[i]);
3691     if (err > max_err) max_err = err;
3692 }
3693 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3694
3695 free(h_a); free(h_b); free(h_c); free(h_c_ref);
3696 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3697 cublasDestroy(handle);
3698 cudaEventDestroy(start);
3699 cudaEventDestroy(stop);
3700 }
3701
3702
3703 static void test_hgemm_mma(int M, int N, int K) {
3704
3705     size_t size_a = (size_t)M * K * sizeof(half);
3706     size_t size_b = (size_t)K * N * sizeof(half);
3707     size_t size_c = (size_t)M * N * sizeof(half);
3708
3709     half *h_a = (half *)malloc(size_a);
3710     half *h_b = (half *)malloc(size_b);
3711     half *h_c_ref = (half *)malloc(size_c);
3712
3713     srand(42);
3714     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3715     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3716

```

```

3717 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
3718 // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
3719 size_t size_b_t = (size_t)N * K * sizeof(half);
3720 half *h_b_t = (half *)malloc(size_b_t);
3721 for (int n = 0; n < N; n++)
3722     for (int k = 0; k < K; k++)
3723         h_b_t[n * K + k] = h_b[k * N + n];
3724
3725 half *d_a, *d_b, *d_b_t, *d_c;
3726 check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
3727 check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
3728 check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
3729 check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
3730
3731 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
3732 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
3733 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
3734
3735 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
3736 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
3737 cublasHandle_t handle;
3738 cublasCreate(&handle);
3739 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3740 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3741              &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
3742              &beta_h, d_c, CUDA_R_16F, N,
3743              CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3744 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
3745
3746 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
3747 cudaEvent_t start, stop;
3748 cudaEventCreate(&start);
3749 cudaEventCreate(&stop);
3750
3751 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
3752 size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
3753 dim3 block(256);
3754 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3755 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
3756 check(cudaGetLastError(), "hgemm launch");
3757 check(cudaDeviceSynchronize(), "hgemm sync");
3758
3759 half *h_c = (half *)malloc(size_c);
3760 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
3761
3762 // Verify
3763 float max_err = 0.0f;
3764 for (int i = 0; i < M * N; i++) {
3765     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3766     if (err > max_err) max_err = err;
3767 }
3768 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
3769
3770 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
3771 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
3772 cublasDestroy(handle);
3773 cudaEventDestroy(start);
3774 cudaEventDestroy(stop);
3775 }
3776
3777
3778 #if defined(NOTES_V2_HAS_WGMMA)
3779 static void test_hgemm_wgmma(int M, int N, int K) {

```

```

3780 // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
3781 // TN layout: C[M×N] = A[M×K] × BT[N×K]
3782 // Kernel: hgemm_wgmma_stages_tn with default template params
3783
3784 constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
3785
3786 // M, K must be divisible by tile dims
3787 if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
3788     printf("| %-35s | %-12s | %-4s |\n",
3789         "HGEMM WGMMMA", "SKIP", "SKIP");
3790     return;
3791 }
3792
3793 size_t size_a = (size_t)M * K * sizeof(half);
3794 size_t size_b = (size_t)K * N * sizeof(half);
3795 size_t size_c = (size_t)M * N * sizeof(half);
3796
3797 half *h_a = (half *)malloc(size_a);
3798 half *h_b = (half *)malloc(size_b);
3799 half *h_c_ref = (half *)malloc(size_c);
3800
3801 srand(42);
3802 for (int i = 0; i < M * K; i++)
3803     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3804 for (int i = 0; i < K * N; i++)
3805     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3806
3807 // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
3808 size_t size_b_t = (size_t)N * K * sizeof(half);
3809 half *h_b_t = (half *)malloc(size_b_t);
3810 for (int n = 0; n < N; n++)
3811     for (int k = 0; k < K; k++)
3812         h_b_t[n * K + k] = h_b[k * N + n];
3813
3814 half *d_a, *d_b, *d_b_t, *d_c;
3815 check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
3816 check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
3817 check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
3818 check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
3819
3820 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
3821 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
3822     "wgmma H2D B (cuBLAS)");
3823 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
3824     "wgmma H2D B_t (kernel)");
3825
3826 // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
3827 cublasHandle_t handle;
3828 cublasCreate(&handle);
3829 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3830 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
3831     CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
3832     CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3833 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
3834     "wgmma D2H ref");
3835
3836 // Create TMA tensor maps for A and BT
3837 // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
3838 // BT[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
3839 CUTensorMap *tma_a =
3840     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
3841 CUTensorMap *tma_b =
3842     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);

```

```

3843 // Launch WGMMMA kernel
3844 // BLOCK_SWIZZLE=false → 2D grid, no swizzle
3845 size_t smem_bytes =
3846     K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
3847 cudaFuncSetAttribute(
3848     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
3849     false>,
3850     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
3851
3852
3853 dim3 block(NUM_THREADS);
3854 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3855 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
3856     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
3857 check(cudaGetLastError(), "wgmma launch");
3858 check(cudaDeviceSynchronize(), "wgmma sync");
3859
3860 half *h_c = (half *)malloc(size_c);
3861 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
3862
3863 // Verify
3864 float max_err = 0.0f;
3865 for (int i = 0; i < M * N; i++) {
3866     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3867     if (err > max_err)
3868         max_err = err;
3869 }
3870 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
3871     max_err < 1.0f ? "PASS" : "FAIL");
3872
3873 free(h_a);
3874 free(h_b);
3875 free(h_b_t);
3876 free(h_c);
3877 free(h_c_ref);
3878 cudaFree(d_a);
3879 cudaFree(d_b);
3880 cudaFree(d_b_t);
3881 cudaFree(d_c);
3882 cudaFree(tma_a);
3883 cudaFree(tma_b);
3884 cublasDestroy(handle);
3885 }
3886 #endif /* NOTES_V2_HAS_WGMMMA */
3887
3888
3889 static void test_flash_attn(int seqlen, int head_dim) {
3890     // FlashAttention-2 with split-Q, MMA m16n8k16
3891     int B = 1, H = 8;
3892
3893     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
3894
3895     srand(42);
3896     half *h_q = (half *)malloc(sz);
3897     half *h_k = (half *)malloc(sz);
3898     half *h_v = (half *)malloc(sz);
3899     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
3900         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3901         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3902         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3903     }
3904
3905     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V

```

```

3906 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
3907 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
3908 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
3909 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
3910 int count = B * H * seqlen * head_dim;
3911 for (int i = 0; i < count; i++) {
3912     ref_q[i] = __half2float(h_q[i]);
3913     ref_k[i] = __half2float(h_k[i]);
3914     ref_v[i] = __half2float(h_v[i]);
3915 }
3916
3917 float scale = 1.0f / sqrtf((float)head_dim);
3918 for (int bi = 0; bi < B * H; bi++) {
3919     for (int qi = 0; qi < seqlen; qi++) {
3920         // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
3921         float smax = -INFINITY;
3922         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
3923         for (int kj = 0; kj < seqlen; kj++) {
3924             float s = 0.0f;
3925             for (int d = 0; d < head_dim; d++)
3926                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
3927                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
3928             S[kj] = s * scale;
3929             if (S[kj] > smax) smax = S[kj];
3930         }
3931         // softmax
3932         double sum_exp = 0.0;
3933         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
3934         float inv_sum = 1.0f / (float)sum_exp;
3935         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
3936         for (int d = 0; d < head_dim; d++) {
3937             double o_acc = 0.0;
3938             for (int kj = 0; kj < seqlen; kj++)
3939                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
3940                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
3941             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
3942         }
3943         free(S);
3944     }
3945 }
3946
3947 half *d_q, *d_k, *d_v, *d_o;
3948 check(cudaMalloc(&d_q, sz), "fa alloc Q");
3949 check(cudaMalloc(&d_k, sz), "fa alloc K");
3950 check(cudaMalloc(&d_v, sz), "fa alloc V");
3951 check(cudaMalloc(&d_o, sz), "fa alloc O");
3952 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
3953 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
3954 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
3955
3956 // Template params for kHeadDim=64, kStage=2
3957 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
3958 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
3959 constexpr int kMmaTileSeqLenQ = 4;
3960 constexpr int kMmaTileSeqLenK = 1;
3961 constexpr int kMmaTileSeqLenP = 4;
3962 constexpr int kMmaTileHeadDimV = 1;
3963 constexpr int kValTileSeqLenQ = 1;
3964 constexpr int kValTileSeqLenK = 8;
3965 constexpr int kValTileSeqLenP = 1;
3966 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
3967
3968 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;

```

```

3969 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
3970 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
3971                     kStageV * Bc * (kHeadDim + kPadV) +
3972                     Bc * (kHeadDim + kPadV)) * sizeof(half);
3973
3974 dim3 block(128);
3975 dim3 grid((seqlen + Br - 1) / Br, B * H);
3976
3977 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
3978    kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
3979    kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
3980    kStageV, kPadV>
3981    <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
3982 check(cudaGetLastError(), "fa launch");
3983 check(cudaDeviceSynchronize(), "fa sync");
3984
3985 half *h_o = (half *)malloc(sz);
3986 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
3987
3988 float max_err = 0.0f;
3989 for (int i = 0; i < count; i++) {
3990     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
3991     if (err > max_err) max_err = err;
3992 }
3993 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
3994
3995 free(h_q); free(h_k); free(h_v); free(h_o);
3996 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
3997 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
3998 }
3999
4000
4001 int main(int argc, char *argv[]) {
4002 #if defined(NOTES_V2_HAS_WGMMA)
4003     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
4004 #endif
4005     int M = 1024, N = 1024, K = 1024;
4006     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
4007
4008     printf("=== notes-v2.cu verification harness ===\n");
4009     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
4010     printf("|-----|-----|-----|\n");
4011
4012     test_block_reduce(N);
4013     test_dot(N);
4014     test_relu(1024);
4015     test_elementwise(1024);
4016     test_histogram(1024);
4017     test_merge_attn_states(512, 16, 128);
4018     test_softmax(256);
4019     test_rms_norm(8, 128);
4020     test_layer_norm(8, 128);
4021     test_rope(8, 128);
4022     test_mat_transpose(256, 256);
4023     test_mat_transpose_padded(256, 256);
4024     test_sgemv(256, 128);
4025     test_sgemm(M, N, K);
4026     test_hgemm_mma(M, N, K);
4027 #if defined(NOTES_V2_HAS_WGMMA)
4028     test_hgemm_wgmma(M, N, K);
4029 #endif
4030     test_flash_attn(1024, 64);
4031

```

```
4032     printf("=== All tests done ===\n");
4033     return 0;
4034 }
4035
4036 // =====
4037 // Quick build & run reference
4038 // =====
4039 // # sm_89 单独编译 + 运行:
4040 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
4041 //
4042 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
4043 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a -DNOTES_V2_HAS_WGMMA -lcublas -lcuda notes-v2.
    cu -o notes_v2_sm90.bin
```